Chapter 7
Object–Process Methodology for Structure–Behavior Codesign

Dov Dori

Abstract Function, structure, and behavior are the three major facets of any system. Structure and behavior are two inseparable system aspects, as no system can be faithfully modeled without considering both in tandem. Object-Process Methodology (OPM) is a systems paradigm and language that combines structure–behavior codesign requirements with cognitive considerations. Based on the formal mathematical foundations of graph grammars and a subset of natural language, OPM caters to human intuition in a bimodal way via graphics and autogenerated natural language text. In a nutshell, OPM processes transform objects by creating them, consuming them, or changing their states. The concurrent representation of structure and behavior in the same, single diagram type is balanced, creating synergy whereby each aspect helps understand the other. This chapter defines and demonstrates the principles and elements of OPM, showing its benefits in facilitating structure–behavior codesign and achieving formal, semantically sound, and humanly accessible conceptual models of complex systems in a host of domains and at virtually any complexity level.

7.1 The Cognitive Assumptions and OPM's Design

Text and graphics are two complementary modalities that our brains process interchangeably. Conceptual modeling, which is recognized as a critical step in architecting and designing systems, is an intellectual activity that can greatly benefit from concurrent utilization of the verbal and visual channels of human cognition. A conceptual modeling framework that employs graphics and text can alleviate cognitive loads [12]. OPM is a bimodal graphics–text conceptual modeling framework that caters to these human cognitive needs. This section argues for the value of the

Dov Dori
Technion, Israel Institute of Technology and Massachusetts Institute of Technology e-mail: dori@ie.technion.ac.il

D. W. Embley and B. Thalheim (eds), *Handbook of Conceptual Modeling.*
DOI 10.1007/978-3-642-15865-0, © Springer 2011

OPM holistic approach in addressing the dual-channel, limited channel capacity, and active processing assumptions. Bimodality, complexity management via hierarchical decomposition, and animated simulation, respectively, address these cognitive needs.

7.1.1 Mayer's Three Cognitive Assumptions

Humans assimilate data and information, converting them into meaningful knowledge and understanding of systems via the simultaneous use of words and pictures. During eons of human evolution, the human brain has been trained to capture and analyze images so it can escape predators and capture food. In contrast, processing of spoken words, let alone text, is the product of a relatively very recent glimpse in the history of mankind. As our brains are hard-wired to process imagery, graphics appeal to the brain more immediately than words. However, words can express ideas and assertions that are far too complex or even impossible to express graphically (try graphing this sentence to get a feeling for the validity of this claim). So while a picture is worth a thousand words, as the saying goes, there are cases where a word, or a sentence, is worth a thousand pictures. A problem with the richness of natural languages is the potential ambiguity that arises from their use. This certainly does not imply that pictures cannot be ambiguous as well, but graphic ambiguity can be greatly reduced, or even eliminated, by assigning formal semantics to pictorial symbols of things and relations among them. Since diagrams usually have fewer interpretations than free text, they are more tractable than unconstrained textual notations.

Mayer [26] found that when corresponding words and pictures are presented near each other, learners can better hold corresponding words and pictures in working memory at the same time, enabling the integration of visual and verbal models. A main contribution of diagrams may be that they reduce the cognitive load of assigning abstract data to appropriate spatial and temporal dimensions. For example, Glenberg and Langston [19] found that where information about temporal ordering is only implicit in text, a flow diagram reduces errors in answering questions about that ordering.

Mayer [26] and Mayer and Moreno [27] proposed a theory of multimedia learning that is based on the following three main research-supported cognitive assumptions.

1. Dual-channel – humans possess separate systems for processing visual and verbal representations [2, 8].
2. Limited capacity – the amount of processing that can take place within each information processing channel is extremely limited [2, 7, 29].
3. Active processing – meaningful learning occurs during active cognitive processing, paying attention to words and pictures, mentally organizing and integrating them into coherent representations. The active-processing assumption is a manifestation of the constructivist theory in education, which puts the construction of

knowledge by one's own mind as the centrepiece of the educational effort [45]. According to this theory, in order for learning to be meaningful, learners must engage in constructing their own knowledge.

7.1.2 Meeting the Verbal–Visual Challenge

As the literature suggests, there is great value in designing a modeling approach and supporting tool that meet the challenges posed by the three cognitive assumptions. While Mayer and Moreno [27] used these assumptions to suggest ways to reduce cognitive overload while designing multimedia instruction, the same assumptions can provide a basis for designing an effective conceptual modeling framework. Indeed, conceptual modeling can be viewed primarily as the active cognitive effort of concurrent diagramming and verbalization of one's thoughts. The resulting diagrams and text together constitute the system's model. A model that is based on a compact set of the most primitive and generic elements is general enough to be applicable to a host of domains and simple enough to express the most complex systems. A sufficiently expressive model can be simulated for detecting design-level errors, reasoning, predicting, and effectively communicating one's design to other stakeholders.

Such a modeling environment would help our brains to take advantage of the verbal and visual channels and to relieve cognitive loads while actively designing, modeling, and communicating complex systems to stakeholders. These were key motivations in the design of OPM [11]. The OPM modeling environment implementation by OPCAT[1] [17] embodies these assumptions. Stateful objects – things that exist in some state – and processes – things that transform objects by changing their state or by creating or destroying them – are the generic building blocks of OPM. Structural and procedural links express static and dynamic relations among entities (objects, object states, and processes) in a system, and a number of refinement/abstraction mechanisms are built into OPM for complexity management.

7.1.3 Dual-Channel Processing and the Bimodality of OPM

Considering the dual-channel assumption, an effective use of the brain is to simultaneously engage the visual and the verbal channels (and hence probably also the two brain hemispheres) for conveying ideas regarding a system's architecture. Indeed, OPM represents knowledge about the system's structure and behavior both pictorially and verbally in a single unifying model. When the user expresses a piece of knowledge in one modality, either graphics or text, the complementary one is automatically updated so the two remain coherent at all times.

[1] An academic individual version of OPCAT for modeling small systems can be obtained from opcat.com.

In order to show how the cognitive assumptions have been accounted for, we follow a stepwise example of bread baking. Figure 7.1 depicts OPCAT's graphical user interface, which displays simultaneously the graphic (top) and textual (bottom) modalities for exploiting humans' dual-channel processing. The top pane presents the model graphically in an object–process diagram – OPD, while the one below it lists the same model textually in Object–Process Language – OPL. OPCAT recognizes OPD constructs (symbol patterns) and generates their OPL textual counterparts. OPL is a subset of natural English. Each OPD construct has a textual OPL-equivalent sentence or phrase. For example, **Baking**, the central system's process, is the ellipse in Fig. 7.1. The remaining five things are objects (the rectangles) that enable or are transformed by **Baking**. **Baker** and **Equipment** are the *enablers* of

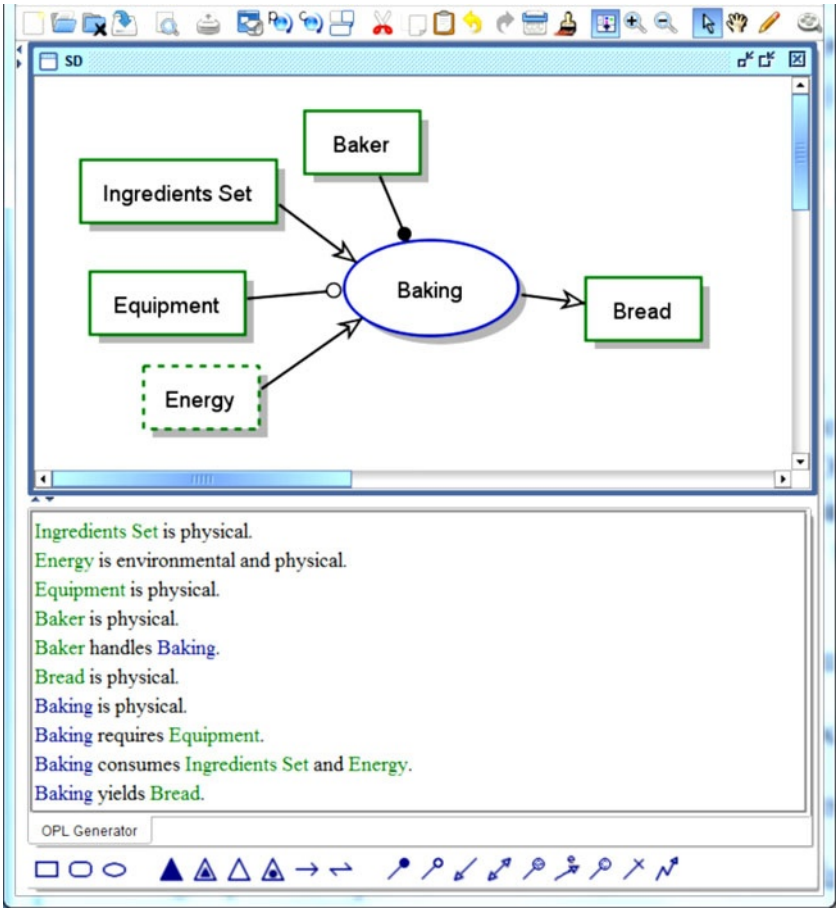


Fig. 7.1 GUI of OPCAT showing the system diagram (SD, top-level diagram) of the Baking system. *Top*: object–process diagram (OPD). *Bottom*: The corresponding, automatically generated Object–Process Language (OPL) paragraph

Baking, while **Ingredients Set**, **Energy**, and **Bread** are its *transformees* – the objects that are transformed by **Baking**. As the direction of the arrows indicates, **Ingredients Set** and **Energy** are the *consumees* – they are consumed by **Baking**, while **Bread** is the resultee – the object created as a result of **Baking**. As soon as the modeler starts depicting and joining things on the graphics screen, OPL sentences start being created in response to these inputs. They accumulate in the OPL pane at the bottom of Fig. 7.1, creating the corresponding OPL paragraph, which tells in text the exact same story that the OPD does graphically.

As the example shows, the OPL syntax is designed to generate sentences in plain natural, albeit restricted, English, with sentences like "**Baking** yields **Bread**." This sentence is the bottom line in Fig. 7.1. An English subset, OPL is accessible to nontechnical stakeholders, and other languages can serve as the target OPL. Unlike programming languages, OPL names can be phrases like **Ingredients Set**.

To enhance the text–graphics linkage, the text colors of the process and object names in the OPL match their colors in the OPD. Since graphics is more immediately amenable to cognitive processing than text, modelers favor modeling the system graphically in the OPD pane, while the textual interpretation is continuously updated in the OPL pane.

The OPL sentences that are constructed or modified automatically in response to linking graphic symbols on the screen provide the modeler and her/his audience with immediate feedback. This real-time humanlike response "tells" the modeler what the modeling environment "thinks" he or she meant to express in the most recent graphic editing operation. When the text does not match the intention of the modeler, a corrective action can be taken promptly. Such immediate feedback is indispensable in spotting and correcting logical design errors at an early stage of the system lifecycle, before they have a chance to propagate and incur costly damage. Any correction of the graphics changes the OPL script, and changes can be applied iteratively until a result that is satisfactory to all the stakeholders from both the customer and the supplier sides is obtained. While generating text from graphics is the prevalent working mode, OPCAT can also generate graphics from text.

The System Diagram (SD, the top-level OPD) is constructed such that it contains a central process, which is the one that carries out the main function of the system, the one that delivers the main value to the beneficiary, for whom the system is built. In our case, **Baking** is the process that provides the value – the **Bread**. This is an example of the application of the *Function as a seed* OPM principle, which states that *modeling of any system starts with specifying the function of the system as the main process in the System Diagram, SD*. Then, objects involved as enablers or transformees are added and both the function and the objects are refined, as described in the sequel. In this system, all the things – objects and processes – are physical, denoted by their shading. **Energy** is also environmental – it is not part of the system, as it is supplied by an external source but is consumed by the system. This is denoted by the dashed contour of **Energy**.

7.1.4 Limited Capacity and the Refinement Mechanisms of OPM

Figure 7.1 is the SD, the bird-eye's view model of the Baking system. This OPD already contains six entities and five links, approaching the limit of the human information processing capacity determined by the "magic number seven plus or minus two" of Miller (1956). However, we have not even started to specify the subprocesses comprising the **Baking** process or the members (parts) of the **Ingredients Set**. To cater to humans' processing limited capacity, OPM advocates keeping each OPD simple enough to enable the diagram reader to quickly grasp the essence of the system by inspecting the OPDs without being intimidated by an overly complicated layout. Overloading the SD with more artifacts will put its comprehension at risk, so showing additional details is deferred to lower-level OPDs, which can be more detailed, as the reader is already familiar with some of their features from upper-level OPDs.

To manage the system's inherent complexity, when an OPD approaches humans' "comprehensibility limit," the model is refined. Refinement in OPD entails primarily the application of the in-zooming refinement mechanism on a process. Figure 7.2 shows a new OPD that resulted from zooming in on the **Baking** process in Fig. 7.1.

This OPD, which is automatically given the name SD1 (**Baking in-zoomed**), elaborates on the SD in several respects. First, **Baking** is inflated, showing within

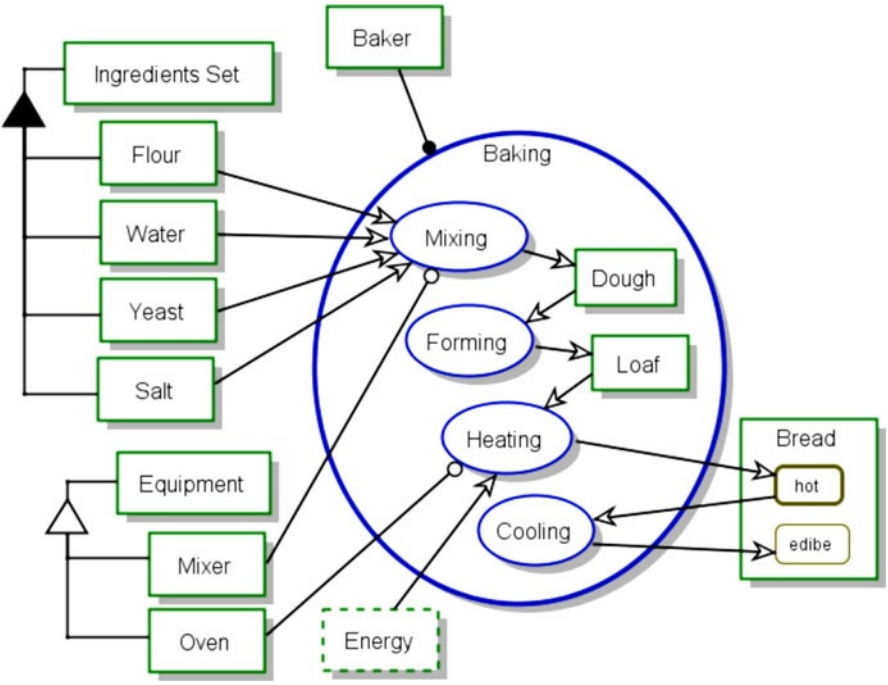


Fig. 7.2 The new OPD resulting from zooming in on the **Baking** process in Fig. 7.1

it the four subprocesses **Mixing**, **Forming**, **Heating**, and **Cooling**, as well as the interim objects **Dough** and **Loaf**. **Bread** is created in the initial state **hot**, and after **Cooling** ends, it is **edible**.

7.1.5 Active Processing and the Animated Simulation of OPM

The active-processing assumption is tacitly accounted for in that each and every modeling step requires complete engagement of the user – the system architect or modeler who carries out the conceptual modeling activity. While modeling, the modeler is active in creating the model, inserting and rearranging elements – entities and links – on the screen. At any time during this process, the modeler can inspect the OPL textual interpretation that is continuously created in response to each new graphic input. At times, he or she needs to rearrange the graphic layout to make it more comprehensible by such actions as grouping entities and moving links to avoid crossings. When the current OPD becomes too busy, it is approaching the limited channel capacity, in which case a new OPD needs to be created via in-zooming or unfolding.

Another aspect of active processing that is unique to OPM is its animated simulation. Humans have been observed to mentally animate mechanical diagrams in order to understand them. Using a gaze tracking procedure, Hegarty [21] found that inferences were made about a diagram of ropes and pulleys by imagining the motion of the rope along a causal chain. Similarly, an active processing aspect of OPCAT is its ability to simulate the system by animating it. The animation enables the modeler to simulate the system and see it "in action" at each point in time during the design. Like a program debugger, the modeler can carry out "design-time debugging" by running the animation stepwise or continuously, back and forth, inserting breakpoints where necessary.

Figure 7.3 is a snapshot of the animated simulation of the **Baking** system, showing it at the point in time when **Heating** just ended, yielding **Bread** at its **hot** state. Currently, as the dots on the arrows to and from **Cooling** indicate, **Cooling** is happening, as indicated by its dark filling. The timeline within an in-zoomed process is from top down, so **Cooling** is the last subprocess of **Baking**, which is therefore dark blue. Rectangular objects (except those with rounded corners) exist at this time point, while white ones (like **Ingredients Set**) are already consumed (or not yet created, but in Fig. 7.3 no such objects exist). The active participation of the modeler in inspecting the system behavior and advancing it step by step has proven highly valuable in communicating action and pinpointing logical design errors, which are corrected early on, saving precious time and avoiding costly troubles downstream.

The ability to animate the system in a simple and understandable manner is yet another benefit of the structure–behavior integration of the OPM model in one type of diagram – the OPD. Splitting the single model into several structural views and several other behavioral views would unnecessarily complicate and obscure the

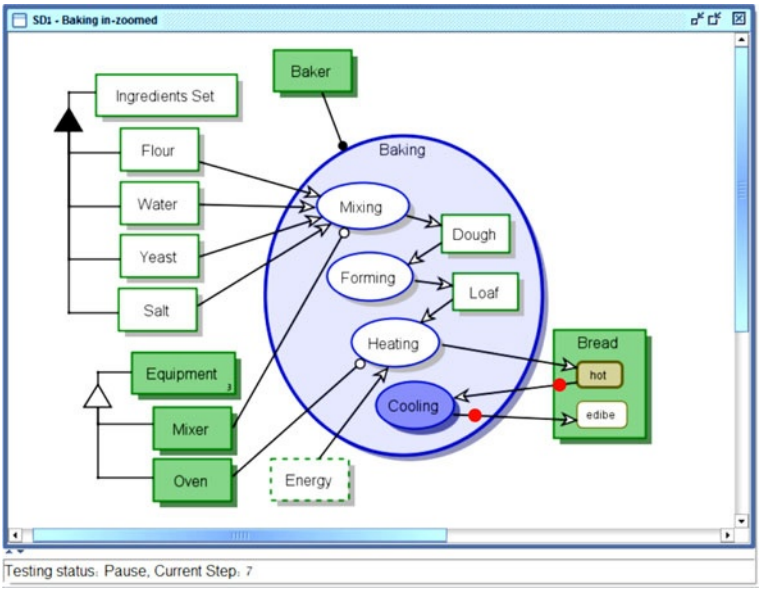


Fig. 7.3 Animated simulation of the OPD in Fig. 7.2

model, preventing it from being amenable to such clear, eye-opening animated simulation.

Having introduced OPM via this simple baking system example, we now define and discuss basic concepts underlying OPM as both a language and a methodology. In parallel, this chapter shows how structure-behaviour codesign is served by the single-model and single-diagram approach of OPM, enabling system architects and designers to freely express the tight, inseparable relationships and interactions between the system's static and dynamic aspects.

7.2 Function, Structure, and Behavior: The Three Major System Aspects

All systems are characterized by three major aspects: function, structure, and behavior. The **function** of an artificial system is its value-providing process, as perceived by the beneficiary, i.e., the person or group of people who gain value from using the system. For example, the function of the organization called hospital is patients' *health level improving*. Each patient is a beneficiary of this system, the customer may be a government or a private entity, and the medical staff constitutes the group of users.

Function, structure, and behavior are the three main aspects that systems exhibit. Function is the top-level utility that the system provides its beneficiaries who use

it or are affected by it, either directly or indirectly. The system's function is enabled by its architecture – the combination of structure and behavior. The system's architecture is what enables it to function so as to provide value to its beneficiaries.

Most interesting, useful, and challenging systems are those in which structure and behavior are highly intertwined and hard to separate. For example, in a manufacturing system, the manufacturing process cannot be contemplated in isolation from its inputs – the raw materials, the model, machines, and operators – and its output – the resulting product. The inputs and the output are objects, some of which are transformed by the manufacturing process, while others just enable it. Due to the intimate relation between structure and behavior, it only makes sense to model them concurrently rather than try to construct separate models for structure and behavior, which is the common practice of current modeling languages like UML and SysML. The observation that there is great benefit in concurrently modeling the system's structure and behavior in a single model is a major principle of OPM.

The **structure** of a system is its form – the assembly of its physical and logical components along with the persistent, long-lasting relations among them. Structure is the static, time-independent aspect of the system. The **behavior** of a system is its varying, time-dependent aspect, its dynamics – the way the system changes over time by transforming objects. In this context, transforming means creating (generating, yielding) a new object, consuming (destroying, eliminating) an existing object, or changing the state of an existing object.

With the understanding of what structure and behavior are, we can define a system's architecture.

The **architecture** of a system is the combination of the system's structure and behavior that enables it to perform its **function**.

Following this definition, it becomes clear why codesign of a system's structure and behavior is imperative: they go hand in hand, as a certain structure provides for a corresponding set of system behaviors, and this, in turn, is what enables the system to function and provide value. Therefore, any attempt to separate the design of a system, and hence its conceptual modeling, into distinct structure and behavior models is bound to hamper the effort to arrive at an optimal design. One cannot design the system to behave in a certain way and execute its anticipated function unless the ensemble of the interacting parts of the system – its structure – is such that the expected behavior is made possible and can deliver the desired value to the beneficiary.

It might be interesting to compare our definition of architecture to that used by the U.S. DoD Architecture Framework (DoDAF 2007), which is based on IEEE STD 610.12:

Architecture: the structure of components, their relationships, and the principles and guidelines governing their design and evolution over time.

The common element in both definitions is the system's structure. However, the DoDAF definition lacks the integration of the structure with the behavior to provide the function. On the other hand, the DoDAF definition includes "the principles and guidelines governing the design and evolution of the system's components over time." However, these do not seem to be part of the system's architecture. Rather, principles and guidelines govern the architecting *process*, which culminates in the system's architecture.

7.2.1 Function vs. Behavior

The above definitions lead to the conclusion that the function of a system is identical with its top-level process. Moreover, the architecture of a system, namely its structure–behavior combination, is what enables the system to execute its top-level process, and thereby to perform its function and deliver value to its beneficiary.

The value of the function to the beneficiary is often implicit; it is expressed in process terms, which emphasize what happens, rather than the *purpose* for which the top-level process happens. This implicit function statement can explain why the function of a system is often confused with the behavior or dynamics of the system. However, it is critical to clearly and unambiguously distinguish between function and behavior. Function is the value of the system to its beneficiaries. The function is provided by operating the system, which, due to its architecture – structure–behavior combination – functions to attain this value. Function explicitly coincides with behavior only at the top-most level. At lower levels, behaviors manifested by subprocesses indirectly serve the function. For example, the electric motor of a water pump in a car with an internal combustion engine functions to circulate water, which in turn cools the engine, which in turn drives the car. The function of the car is driving, but processes at various levels of granularity are required to make this happen.

This distinction between function and behavior is of utmost importance since in many cases a system's function can be achieved by different architectures, i.e., different combinations of processes (system behavior) and objects (system structure). Consider, for example, a system for enabling humans to cross a river with their vehicles. Two obvious architectures are ferry and bridge. While the two systems' function and top-level process – river crossing – are identical, they differ dramatically in their structure and behavior. Similarly, a time-keeping system can be a mechanical clock, an electronic watch, or a sundial, to name a few possible system architectures, each with its set of "utilities" (availability, maintainability, precision). Failure to recognize this difference between function and behavior may lead to a premature choice of a suboptimal architecture. In the river-crossing system example above, this may amount to making a decision to build a bridge without considering the ferry option altogether.

Capturing the knowledge about existing systems and the analysis and design of conceived systems requires an adequate methodology, which should be both formal and intuitive. Formality is required to maintain a coherent representation of the system under study, while the requirement that the methodology be intuitive stems from

the fact that humans are the ultimate consumers of the knowledge. OPM [10, 11] is an ontology- and systems-theory-based vehicle for codesign via conceptual modeling, as well as for knowledge representation and management, which perfectly meets the formality and intuition requirements through a unique combination of graphics and natural language.

Modeling of complex systems should conveniently combine structure and behavior in a single model. Motivated by this observation, OPM is a comprehensive, holistic approach to the modeling, study, development, engineering, evolution, and lifecycle support of systems. Employing a combination of graphics and a subset of English, the OPM paradigm integrates the object-oriented, process-oriented, and state-transition approaches into a single frame of reference that is expressed in both graphics and text. Structure and behavior coexist in the same OPM model without highlighting one at the expense of suppressing the other to enhance the comprehension of the system as a whole.

A systems modeling methodology and language must be based on a solid, generic ontology. The next section discusses this term as an introduction to presenting the OPM ontology that follows.

7.2.2 Ontology

Ontology is defined as a branch of philosophy that deals with modeling the real world [48]. Ontology discusses the nature and relations of being, or kinds of existence [31]. More specifically, ontology is the study of the categories of things that exist or may exist in some domain [40]. The product of such a study, called ontology, is a catalog of the types of things that are assumed to exist in a domain of interest from the perspective of a person who uses a specific language, for the purpose of talking about that domain.

The traditional goal of ontological inquiry is to discover those fundamental categories or kinds that define the objects of the world. The natural and abstract worlds of pure science, however, do not exhaust the applicable domains of ontology. There are vast, human-designed and human-engineered systems, such as manufacturing plants, hospitals, businesses, military bases, and universities, in which ontological inquiry is just as relevant and equally important. In these human-created systems, ontological inquiry is primarily motivated by the need to understand, design, engineer, and manage such systems effectively. Consequently, it is useful to adapt the traditional techniques of ontological inquiry in the natural sciences to these domains as well [23].

The types in the ontology represent the predicates, word senses, or concept and relation types of the language L when used to discuss topics in the domain D. An uninterpreted logic, such as predicate calculus, conceptual graphs, or Knowledge Interchange Format [22], is ontologically neutral. It imposes no constraints on the subject matter or the way the subject may be characterized. By itself, logic says nothing about anything, but the combination of logic with ontology provides a language about the entities in the domain of interest and relationships among them.

An *informal ontology* may be specified by a catalog of types that are either undefined or defined only by statements in a natural language. In every domain, there are phenomena that the humans in that domain discriminate as (conceptual or physical) objects, processes, states, and relations. A *formal ontology* is specified by a collection of names for concept and relation types organized in a partial ordering by the generalization–specialization (also referred to as the type–subtype or class–subclass) relation. Formal ontologies are further distinguished by the way the subtypes are distinguished from their supertypes. An *axiomatized ontology* distinguishes subtypes by axioms and definitions stated in a formal language, such as logic; a *prototype-based ontology* distinguishes subtypes by a comparison with a typical member or prototype for each subtype. Examples of axiomatized ontologisms include formal theories in science and mathematics, the collections of rules and frames in an expert system, and specifications of conceptual schemas in languages like SQL. OPM concepts and their type ordering are well defined; hence OPM belongs to the family of axiomatized ontologies.

The IDEF5 method [23] is designed to assist in creating, modifying, and maintaining ontologies. Ontological analysis is accomplished by examining the vocabulary that is used to discuss the characteristic objects and processes that compose a domain, developing definitions of the basic terms in that vocabulary, and characterizing the logical connections among those terms. The product of this analysis, ontology, is a domain vocabulary complete with a set of precise definitions, or axioms, that constrain the meanings of the terms sufficiently to enable consistent interpretation of the data that use that vocabulary.

Rather than requiring that the modeler view each of the system's aspects in isolation and struggle to mentally integrate the various views, OPM offers an approach that is orthogonal to customary practices. According to this approach, the structure and behavior system aspects can be inspected in tandem rather than in isolation, providing for better comprehension, as argued above. Complexity is managed by the ability to create and navigate via possibly multiple detail levels, which are generated and traversed through two refinement/abstraction mechanisms: in-zooming/out-zooming and unfolding/folding.

OPM strives to be as generic as possible. Hence its domain is the universe, and its building blocks are things – objects and processes, defined below. Due to its structure–behavior integration, OPM provides a solid basis for representing and managing knowledge about complex systems, regardless of their domain. The remainder of this chapter provides an overview of OPM, its ontology, semantics, and symbols. It then describes applications of OPM in various domains.

7.3 The OPM Ontology

The elements of the OPM ontology, shown in Fig. 7.4, are divided into three groups: entities, structural links, and procedural links.

7.3.1 Entities: Objects, Processes, and Object States

Entities, the basic building blocks of any system modeled in OPM, are of three types: stateful objects, namely, *objects* with *states*, and *processes*. Each entity type stands alone as a concept in its own right, unlike links, which connect two entities. The symbols for these three entities are respectively shown as the first group of symbols on the left-hand side of Fig. 7.4, which contains the symbols in the toolset available as part of the GUI of OPCAT.

Objects are our way of knowing what *exists*, or in other words, the *structure* of systems. To know what *happens*, to understand systems' behavior, a second, complementary type of thing is needed – processes. We know of the existence of an object if we can name it and refer to its unconditional, relatively stable existence, but without processes we cannot tell how this object changes over time.

Objects and processes, collectively referred to as OPM *things*, are the two types of OPM's universal building blocks. OPM views objects and processes as being on equal footing, so processes are modeled as "first-class citizens" that are not subordinate to objects. This is a principal departure from the object-oriented (OO) approach, which places objects as the only major players, and they "own" processes, which in OO jargon are called operations or services or methods.

Major system-level processes can be as important as, or even more important than, objects in the system model. In particular, we already noted that the top-level process of a system is its function, the objective for which the system is built and used. Hence, processes must be amenable to being modeled independently of a particular object class. In OPM, both objects and processes are first-class citizens. They stand on equal footing, such that neither one has supremacy over the other. This OPM *object–process equality principle* enables OPM to model real-world systems in a single model, in which structure and behavior are concurrently represented and can be codesigned. Due to this structure–behavior unification, the single OPM model is simple and intuitive in spite of its formality. The third and last OPM entity, after object and process, which are OPM things, is state. A state, discussed in more detail below, is a situation in which an object can be at some point during its lifetime.

Being able to tell objects and processes apart and use them properly in a model is a key to modeling in OPM. To define these fundamental concepts and to communicate their semantics, we next discuss the concepts of existence and transformation.

Fig. 7.4 The three groups of OPM symbols

7.4 Existence, Things, and Transformations

Webster's New World College Dictionary [50] defines *existence* as the noun derived from exist, which is *be, have being, continue to be*. To exist means to stand out, to show itself, and have an identifiable, distinct uniqueness within the physical or mental realm. A thing that exists in physical reality has "tangible being" at a particular place and time. Because it stands out and shows itself, we can point to it and say: "Now, there it is."

That which we can never identify, or have its identity be inferred in some way, can have no existence for us. In other words, "to stand out" requires a continuous identifiability over an appropriate duration of time, either physically or informatically, as we elaborate below.

When we consider existence along the *time* dimension, there are two modes of "standing out," or existence of things. In the first mode, the "standing out" takes place during a positive, relatively substantial time period. This "standing out" needs to be observable in a form that is basically unchanging, stable, or persistent. We call that which stands out in this mode an *object*.

7.4.1 Physical and Informatical Objects

Webster's 1997 Dictionary [50] defines an object as follows.

[An object is] a material thing; that to which feeling or action is directed; end or aim; word dependent on a verb or preposition.

Webster's 1984 Dictionary [49] provides a different set of relevant definitions for object:

- *Anything that is visible or tangible and is stable in form.*
- *Anything that may be apprehended intellectually.*

These two definitions correspond to our notion of a *physical* object and an *informatical object*. The first definition is the one we normally think of when using the term object in daily usage. The second definition pertains to the informatical, intangible facet of objects. Informatical objects are different from their physical counterparts in that informatical objects have no physical existence and, being intangible, they do not obey the basic laws of physics. However, the *existence* of informatical objects does depend on their being symbolically recorded, inscribed, impressed, or engraved on some tangible medium. This information-carrying medium is a physical object. It can be a stone, papyrus, paper, some electromagnetic medium, or the human brain.

7.4.2 Object Defined

Since OPM uses objects that are physical or informatical, we define object as something that captures these two facets without committing to either one, while including the element of "existence throughout time."

*An **object** is a thing that exists or can exist physically or informatically.*

This definition is quite remote from the classical definitions of object found in the OO literature, which can be phrased as follows: "An object is an abstraction of attributes and operations that is meaningful to a system." For example, in Wikipedia, an object in computer science is defined as a language mechanism for binding data with methods that operate on those data.

A process, on the other hand, is typically a *transient, temporal* thing. It "happens" or "occurs" to an object rather than something that "exists" in its own right. We cannot think of a process independently of at least one object, the one that the process transforms. By their nature, happenings or occurrences involve time. What actually exists as a process is an informatical object that represents the *pattern of behavior*, which the objects that are involved in the interaction exhibit as time goes by. This idea is elaborated on next.

7.4.3 Process as a Transformation Metaphor

There are two perspectives from which a system can be contemplated. One perspective is an instantaneous, snapshotlike, structural one, which views the world at some moment of time. This perspective is timeless; it has no time dimension. The structural perspective represents objects and time-independent relationships that may exist among them. This perspective has no room for processes, as by definition they occur over time, which is not considered in this view. A second perspective is the temporal one, where time is considered as a central theme, such that time-dependent relationships among things are representable. From this temporal viewpoint, the existence of an object is persistent. As long as the object is not involved in a process, it remains unchanged.

We noted that there are two modes of standing out. The first is in space, the second – in time. In the time mode, that which stands out is changing and may have different names as it undergoes its transformation. In particular, the name given to what stands out after the change is complete may be different from the name the thing had before the change occurred. In this case, it is convenient to think of the thing – a process – that has brought about a transformation as some carrier that is "responsible" for this transformation. Hence, the concept of *process* is our abstraction of the thing behind the series of transformations that one or more objects

undergo. For the convenience of language or thinking, we associate this patterned changing with the "carrier," to which we mentally assign the "responsibility."

We define *transformation* as a generalization of change, generation, and destruction of an object.

Transformation is the generation (construction, creation) or consumption (destruction, elimination) or change (effect, state transition) of an object.

When we say that the process brought about the creation (or generation or construction) of an object, we mean that the object, which had not existed prior to the occurrence of the process, now exists and is identifiable against its background. Analogously, when we say that the process brought about the elimination (or consumption or destruction) of an object, we mean that the object, which once stood out, has undergone a radical change, due to which it no longer exists. These radical changes of creation and elimination are extreme versions of transformation. Processes are the only things that cause creation and elimination of objects as well as changes in the objects' states. Collectively, creation, elimination, and change are termed transformation.

We call the carrier that causes transformation *process*, and we say that the process is the thing that brings about the transformation of an object. However, that carrier is just a metaphor, as we cannot "hold" or touch a process, although that process may be entirely physical, such as filling a glass of water. What we may be able to touch, see, or measure at given points in time is the object, or one or more of its *attributes*, as the process transforms the object. For example, we can see the glass and the water being poured into it, gradually changing the "fullness" attribute of the glass from empty to full.

At any given point in time before, during, or after the occurrence of the process, the observed object can potentially be different from what it was at a previous point in time. Using our human memory, we get the sense of a process by comparing the present form of the object being transformed to its past form. Hence, we can almost say that *a process is only in a human's mind*, as only through comparing objects or their states at various points in time can we tell that a process took place. Unlike objects, processes do not exist; they happen or occur. The only possible existence of processes is in our minds, where they are recorded as informatical abstract concepts of patterns of object transformations.

7.4.4 Process Defined

According to Webster's 1997 Dictionary [50], a process is "a state of going on, series of actions and changes, method of operation, action of law, outgrowth." The American Heritage Dictionary [1] defines process as "a series of actions, changes, or functions, bringing about a result." Focusing on transformation and the result

or effect that it induces, and based on the observations noted above, we adopt the following definition.

*A **process** is a transformation that an object undergoes.*

This definition of process acting on an object immediately implies that no process exists unless it is associated with at least one object, which the process transforms. This is where the symmetry between objects and processes breaks. While we could refer to an object without necessarily using the term "process," the opposite is not possible – the ability to conceive a process depends on the existence of at least one object, which undergoes transformation due to that process. Another asymmetry between an object and a process is related to states. Whereas objects can be stateful, processes do not have states. What can be thought of as states of a process are its possible subprocesses.

The transformation of an object takes place over some time. It may be as minor as moving the object from one location to the other, or as drastic as destroying or creating the object. Without processes, all we can describe are static, persistent structural relations among objects. In a theoretic, frozen, static universe at absolute zero temperature, no processes of interest occur. In a more realistic setting, processes and objects are of comparable importance as building blocks in the description and understanding of systems and the universe as a whole.

As an example, consider Newton's first law, which can be formulated as "Every object persists in a state of rest or uniform motion in a straight line unless compelled by an external force to change that state" [44]. Here, the object is physical, and the "external force" is the process that acts on the object to change its state. The state can be either "moving at constant speed" (including "resting," which amounts to "moving at constant speed that is equal to zero") or "moving at variable speed." As long as no process acts on the object, the object persists in its current state of "moving at constant speed."

7.4.5 Cause and Effect

One insight from investigating the time relationship is *cause* and *effect*. Certain objects, when brought into the right spatial and temporal relationship, enable a process to take place. This is the *preprocess object set* – the *precondition* for the process occurrence. For the process to actually happen, it needs to be triggered. If it is triggered and all the preconditions are met, the process occurs. When the process is over, at least one of the objects involved (as input, output, or both) is transformed (consumed, generated, or changed). The "cause" is the trigger – the event that took place in the concurrent presence of the collection of objects (some of which might need to be in a certain state) that enabled the process – the object transformation. The transformation is the "effect." For example, the process of running an internal

combustion engine is contingent upon the presence of the object air–gasoline vapor mixture, at the right pressure and temperature (which are attributes of the mixture), inside the object cylinder, and the concurrent presence of a spark (created by a previous timed process) – the trigger that ignites the mixture. As a result of this process, the gasoline mixture is consumed and the blast increases the pistons' kinetic energy value. If the spark – the trigger – is timed before the mixture is ready or after is has dissipated, no process takes place. In general, the trigger has to be timed such that the process precondition is met; otherwise the trigger has no influence on the system.

7.5 Syntax vs. Semantics

To make it possible to refer to things (both objects and processes) and distinguish among them, natural languages assign names to them. The name of a thing constitutes a primary identifying symbol of that thing, making it amenable to reference and communication among humans. These thing names are known as *nouns*. However, being part of speech, noun is a syntactic term, while objects and processes are semantic terms. We elaborate on this issue next.

7.5.1 Objects to Semantics Is Like Nouns to Syntax

In natural languages, both objects and processes are syntactically represented as nouns. Thus, for example, both "brick" and "construction" are nouns. However, brick is an object, while construction is a process. This can be verified by the fact that the phrase *the construction process* is plausible, while *the brick process* is not. Analogously, the phrase *the object brick* is plausible, while *the object construction* is much less plausible. Natural languages are often even more confusing in this regard. For example, the object building (house, edifice, a noun), which is the outcome of the building (construction) process, is spelled and uttered the same as the process of building (verb). It is only from their context inside a sentence that these two semantically different words – building and building – are distinguishable. These examples demonstrate that we need a semantic, content-based analysis to tell objects from processes.

While in many natural languages nouns primarily represent objects, not every noun is an object, as the examples above demonstrate. Thus, while *construct* is an object, *construction* is a process, yet both are nouns. This is a source of major confusion. We must be aware of this distinction between object and noun and be able to tell apart nouns that are objects from nouns that are processes.

7.5.2 Syntactic vs. Semantic Sentence Analysis

The difficulty we often experience in making the necessary and sufficient distinction between objects and processes is rooted in our education. It is primarily due to the fact that as students in high school we have been trained to think and analyze sentences in syntactic terms of parts of speech: nouns, verbs, adjectives, and adverbs, rather than in semantic terms of objects and processes.

This is probably true for any natural language we study and use, be it our mother tongue or a foreign language. Only through *semantic sentence analysis* can we overcome superficial differences in expression and get down to the intent of the writer or speaker of some text. Nevertheless, the idea of semantic sentence analysis, in which we seek the deep meaning of a sentence beneath its appearance, is probably a relatively less accepted idea. Sentences in Object–Process Language (OPL) are constructed automatically in response to the OPM modeler's graphic input. These sentences have a simple syntax that expresses unambiguous semantics. In OPL, **bold Arial font** denotes nonreserved phrases, while **nonbold Arial** font denotes reserved phrases. In OPCAT, various OPM elements are colored with the same color as their graphic counterparts (by default, objects are rectangular, processes are oval, and states are rectangular with rounded corners).

7.6 The Process Test

To apply OPM in a useful manner, one should be able to tell the difference between an object and a process. The *process test*, defined in this section, has been devised to help identify nouns that are processes rather than objects. Its importance lies in the fact that it is very instrumental in helping analysts to make the essential distinction between objects and processes, a prerequisite for successful system analysis, modeling, and design. Providing a correct answer to the question "Is noun X an object or a process?" is crucial and fundamental to the entire object–process paradigm. The *object–process distinction problem* is simply stated as follows: *Classify a given noun as either an object or a process*.

By default, a noun is an object. To be a process, the noun has to pass the following process test. A noun passes the process test if and only if it meets each of the four *process criteria*:

- Object involvement,
- Object transformation,
- Association with time, and
- Association with verb.

7.6.1 The Preprocess Object Set and Object Involvement

The first process criterion, *object involvement*, implies that in order for the noun in question to be a process, i.e., to happen, a set of one or more nouns, which would be objects, some possibly in certain required states, must be involved.

*The process test **object involvement criterion** is satisfied if the occurrence of the noun in question involves at least one other noun, which would be an object.*

If the noun is indeed a process, the objects that need to exist for the process to happen constitute the *preprocess object set* of that process, as defined below.

*The **preprocess object set** of a process is the set of one or more objects that are required to simultaneously exist, possibly in certain states, in order for that process to start executing once it has been triggered.*

Existence of all the objects in this preprocess object set, possibly in their required states, is the *process precondition* – the condition for the occurrence of the process. As a process, this noun does not exist, but rather occurs, happens, operates, executes, transforms, changes, or alters at least one other noun, which would be an object.

Let us consider two process examples: **Flight** and **Manufacturing**. For **Manufacturing**, the preprocess object set may consist of **Raw Material**, **Operator**, **Machine**, and **Model**, without which no **Manufacturing** can occur. In the **Flight** example, **Airplane**, **Pilot**, and **Runway** are objects in the preprocess object set, since **Flight** cannot occur without them. Moreover, there may be requirements for the state of each of these objects. For **Flight** to take off, it is required that **Airplane** be **operational**, **Pilot** be **sober**, and **Runway** be **open**.

7.6.2 The Postprocess Object Set and Object Transformation

The postprocess object set is defined analogously to the preprocess object set as follows.

*The **postprocess object set** of a process is the set of one or more objects that simultaneously exist, possibly in certain states, after that process has finished executing.*

Existence of all the objects in the postprocess object set, some possibly in specified states, is the *postcondition* of that process.

The preprocess object set and the postprocess object set are not necessarily disjoint; they can be overlapping. In the **Flight** example, all three objects in the preprocess object set, **Airplane**, **Pilot**, and **Runway**, are also in the postprocess object set. However, only **Airplane** and **Pilot** are transformed, as their **Location** attributes change from **source** to **destination**. In the **Manufacturing** example, **Raw Material**, **Operator**, **Machine**, and **Model** are in the preprocess object set, while **Operator**, **Machine**, **Model**, and **Product** are in the postprocess object set. Here, **Raw Material** is transformed by being consumed, while **Product** is transformed by being created.

The second process criterion, *object transformation*, stipulates that a process must transform at least one of the objects in the preprocess object set or in the postprocess object set. In other words, at least one object from the preprocess object set or the postprocess object set must be *transformed* (consumed, created, or change its state) as a result of the occurrence of the noun in question.

*The process test **object transformation criterion** is satisfied if the occurrence of the noun in question results in the transformation of at least one other noun, which would be an object.*

As noted, the transformed object can belong only to the preprocess object set (if it is consumed), only to the postprocess object set (if it is created), or to both (if it is affected, i.e., if its state changes).

Continuing the two previous process examples, **Flight** transforms **Airplane** by changing its **Location** attribute from **origin** to **destination**. **Manufacturing** transforms two objects: **Raw Material** (by consuming it) and **Product** (by generating it). Here, only **Raw Material** is a member of the preprocess object set, and only **Product** is a member of the postprocess object set.

7.6.3 Association with Time

The third process criterion, *association with time*, is that the process must represent some happening, occurrence, action, procedure, routine, execution, operation, or activity that takes place along the timeline.

*The process test **association with time** criterion is satisfied if the noun in question can be thought of as happening through time.*

Continuing our example, both **Flight** and **Manufacturing** start at a certain point in time and take a certain amount of time. Both time and duration are very relevant features of these two nouns in question.

7.6.4 Association with Verb

The fourth and last process criterion, *association with verb*, requires that a process be associated with a verb.

> *The process test **association with verb** criterion is satisfied if the noun in question can be derived from, or has a common root with, a verb.*

Flying is the verb associated with **Flight**. The sentence "The airplane flies" is a short way of expressing the fact that the **Airplane** is engaged in the process of **Flight**. Similarly, manufacture (produce, yield, make, create, generate) is the verb associated with **Manufacturing**. The sentence "The operator manufactures the product from raw material using a machine and a model" is the natural language short way of saying in OPL that:

Operator handles Manufacturing.
Manufacturing requires Machine and **Model**.
Manufacturing yields Product.

It is not mandatory that the verb be syntactically from the same root as the process name, as long as the semantics is the same. For example, **Marrying** is a process, which is associated with the verb marry. Wed is also a legal verb, albeit less frequently used. Alternatively, we could use **Wedding** to fit it to the verb wed.

Many objects, such as **Apple** and **Airplane**, are not associated with any verb, so they do not fulfill this process criterion. It is easy to verify that both **Apple** and **Airplane** do not meet any one of the previous three process criteria either. As noted, however, failure to fulfill even one of the four criteria results in failure of the entire process test.

7.6.5 Boundary Cases of Objects and Processes

While objects are persistent (i.e., they exhibit static perseverance, or, in other words, the value of their **Perseverance** attribute is **static**) and processes are transient (i.e., they exhibit dynamic perseverance, or, in other words, the value of their **Perseverance** attribute is **dynamic**), boundary examples of persistent processes and transient objects exist.

7.6.5.1 Persistent, State-Preserving Processes

Persistent processes are state-preserving processes. They act to preserve, maintain, or keep a steady state or a status quo of a system. They include such verbs as *holding, maintaining, keeping, staying, waiting, prolonging, delaying, occupying, persisting,*

preventing, including, containing, continuing, supporting, and *remaining*. Rather than induce any real change, the semantics of these verbs leaves the state of the object as is, in its status quo, for some more time.

*A **persistent process** is a pseudo process that acts to preserve, maintain, or keep a steady state or a status quo of the system.*

Strictly speaking, these are not real processes, as they fail the object transformation criterion of the process test. Indeed, the static nature of these verbs is contradictory to the definition of process, which requires that it transform some object. In fact, the process test for "**Supporting**" in a system in which a pedestal supports a statue fails because it does not meet the object transformation criterion, as no object is transformed by the support.

Surrounding in the context of a **Highway** that surrounds a **City** is another example. **Surrounding** in this context is not a real process – it too does not meet the object transformation criterion, as no object is transformed by it. Such cases are actually specifications of some structural relation between two objects and are therefore better modeled using tagged structural links, defined and described in the sequel. For example, **surrounds** is a tagged structural relation, from which the OPL sentence "**Highway surrounds City**" is derived.

From another perspective, some of the verbs expressing state preservation can be considered as working against some "force" that would otherwise change some object. For example, a **Pedestal** supporting a **Statue** works against gravity, so we can think of **Supporting** as a "falling-prevention" process, without which the state of the **Statue** would change from **stabilized** to **fallen**.

7.6.5.2 Transient Objects

A *transient object* is a short-lived object that exists only in the context of some subprocess in a system model. Examples of transient physical objects are unstable materials or particles, such as an interim short-lived compound in a chemical reaction or an atom in an excited state that spontaneously decays to the ground state by emission of X-rays and fluorescent radiation [3]. Examples of transient informatical objects are pointers to memory locations in a database system or typed passwords in a Web-based system that are created and immediately deleted after being used.

*A **transient object** is an object that exists only in the context of some process in a system model.*

The context of the transient object is the process within which it is created and consumed. For example, a packet of data in a telecommunications network is a transient informatical object that lives only in the context of the particular instance of

a data communication process. Such a packet can reside for a short while at some router on its way and leave no trace of its stay there once the target node has received it and assembled it into a file from which it was derived.

In an OPM model, a transient object that is created by some subprocess within a process and is immediately consumed by a subsequent subprocess can be skipped by using the *invocation link*. This lightening-shaped link is a procedural link that directly connects the two processes, as discussed in the sequel.

7.6.6 Thing Defined

We have seen that objects exist while processes occur. Objects are relatively persistent, static things, while processes are transient, dynamic things. Objects cannot be transformed (generated, affected, or eliminated) without processes, while processes have no meaning without the objects they transform. Hence, objects and processes are two types of tightly coupled complementary *things*. The extent of this coupling is so intense that if we wish to be able to analyze and design systems in any domain as intuitively and naturally as possible, we must consider objects and processes concurrently. While this point has already been made more than once in this chapter, it is worth repeating.

Given a system model in which objects and processes are described in tandem, we are immediately able to tell the set of objects that are transformed by each process and the nature of the transformation of each transformed object – generation, consumption, or state change. Moreover, we are able to tell how refineables – parts, features, or specializations of these objects, discussed later – are involved in subprocesses of these process. This is the extent to which objects and processes are interwoven and the corresponding value of modeling them concurrently.

As we shall see, objects and processes have much in common in terms of relations such as aggregation, generalization, and characterization. The need to talk about a generalization of these two concepts necessitates the advent of a term that is more abstract than an object and a process, and which generalizes the two. As we have already seen, we call this concept simply a "thing."

Thing is a generalization of object and process.

The concept of "thing" enables us to think and to express ourselves in terms of this abstraction and refer to it without the need to repeatedly iterate the words "object or process." The term "thing" is based on the ontology of Bunge [5, 6] and Wand and Weber [46, 47]. Their first premise is that *the world is made of things that have properties*. According to this definition, *thing* seems to be synonymous with the OPM notion of *object*. However, during the last two decades, the term "object" has become deeply rooted, at least in the software engineering community, with the object-oriented concept of object as an encapsulation of attributes and operations. In

SysML, object has been replaced by "block." It therefore seems justified to extend the semantics of *thing* from an object to the generalization of object and process.

7.6.7 States

Objects can be stateful, i.e., they may have one or more states.

> *A **state** is a situation in which an object can exist or a value it can assume at certain time intervals.*

Since a state is always related to or "owned by" an object, it can be found only inside an object. Unlike OPM things – objects and processes – which, as argued, stand on equal footing, a state is a tad lower in this hierarchy, as by definition it is affiliated with the object that owns it.

A stateful object, i.e., an object that has states, can be *affected* by a process. In that case, the affecting process changes the current state of the stateful object.

> **Effect** is a change in the state of an object.

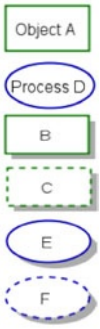
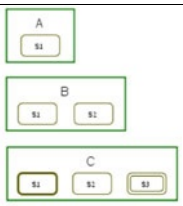
A process that affects an object changes the state of that object from the input (initial) state to the output (final) state.

> **Input state** is the state in which the object being affected is at the beginning of the affecting process.
>
> **Output state** is the state at which the object being affected is at the end of the affecting process.

An object can be in some of its states (or assume one of its values), but it can also be in transition between two of its states. This is the case when a process is transforming the object, such that the object is no longer in its input state and is not yet at its output state.

Table 7.1 presents the symbols and descriptions of states and values. A value is in principle no different than a state. However, we refer to a situation as a state when there is a small number of discrete situations and they are of an enumerated type. We refer to it as a value when there is a large number of such situations and they are often numeric and continuous.

Table 7.1 Entities of the OPM ontology and **Thing** variants due to different **Essence** and **Affiliation** attributes

ENTITIES				
	Name	**Symbol**	**OPL**	**Definitions**
Things	**Object**		**B** is physical. *(shaded rectangle)* **C** is physical and environmental. *(shaded dashed rectangle)* **E** is physical. *(shaded ellipse)* **F** is physical and environmental. *(shaded dashed ellipse)*	An **object** is a thing that exists. A **process** is a thing that transforms at least one object. Transformation is object generation or consumption, or effect—a change in the state of an object.
	Process			
	State		**A** is **s1**. **B** can be **s1** or **s2**. **C** can be **s1**, **s2**, or **s3**. **s1** is initial. **s3** is final.	A **state** is situation an object can be at or a value it can assume. States are always within the object that owns them. A state can be initial, final, or both.

7.6.8 Things and States Are Entities, Entities and Links are Elements

The word **Thing** may sound too mundane, so one might wish to use the more sophisticated word "entities" instead. However, an entity tends to be interpreted more statically than dynamically, i.e., more as an object than as a process, while we wish to use a word that is as neutral and abstract as possible, so "thing" is preferable. However, we do use "entity" when we wish to refer collectively to things and states. As we have seen, a state is a situation in which an object can be. As such, the semantics it conveys is also static. OPM therefore uses the term **Entity** to generalize a **Thing** and a **State**.[2]

Entity is a generalization of thing and state.

Things (objects and processes) and states are collectively called entities. Since a state can only exist within an object, a stateful object with n distinct states can be thought of as n distinct stateless objects, each with its state name preceding the

[2] As we shall see, the meanings of Value and State are very close. In this chapter, we use value and state interchangeably.

name of the stateful object. For example, the stateful object **Car**, which can be in two states, **operational** and **broken**, is equivalent to two stateless objects: **Operational Car** and **Broken Car**.

Climbing one level higher in this hierarchy, links connect entities. Collectively, links and entities are OPM elements.

Element is a generalization of entity and link.

Element is highest in the hierarchy. To specify this hierarchy formally, we use a reflective metamodel, described next.

7.7 A Reflective Metamodel of OPM Elements

We have already defined a substantial part of OPM, but so far we have only done so in text, contrary to the claim that there is great value in formal graphic modeling. This will change right now, as we start modeling the language of OPM. A model-based representation of a modeling language is done via metamodeling it to create a metamodel – a model that specifies the modeling language.

*A language **metamodel** is a specification of a modeling language by a modeling language.*

If the modeling language used to create the metamodel is the same as the modeling language being specified, then the metamodel is a "self metamodel" or a reflective metamodel [34].

*A **reflective metamodel** is a specification of a modeling language by itself.*

7.7.1 An Initial OPM Reflective Metamodel

Figure 7.5 is a portion of a reflective metamodel of the OPM elements hierarchy. At the top of the hierarchy is **OPM Thing**, which, in turn, specializes into **Object** and **Process**. The *generalization–specialization* ("is-a") link, which is a *structural* link, is symbolized graphically as an empty isosceles triangle whose tip is linked to the general thing and whose bottom – to the specialized thing(s).

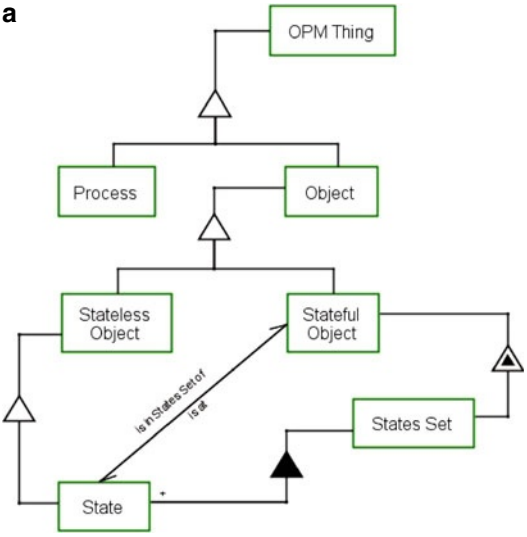


Fig. 7.5 An initial OPM reflective metamodel of the OPM elements hierarchy. **a** Object–process diagram (OPD). **b** Corresponding Object–Process Language (OPL) paragraph

7.7.2 The OPM Graphics–Text Equivalence Principle

Any OPM model is represented concurrently in two modalities: graphics and text. The graphics–text equivalence OPM principle is as follows.

> *In any OPM system model, each OPD – the graphic modality – has an equivalent OPL paragraph – the textual modality, such that both contain exactly the same information and are therefore reconstructible from each other.*

Figure 7.5b contains the OPL paragraph that is the textual representation of the OPD in Fig. 7.5a. The OPM graphics–text principle can be verified here. For example, the first OPL sentence is "**Object is an OPM Thing**." This is the textual equivalence of the generalization–specialization link from **OPM Thing** to **Object**.

7.7.3 The Five Basic Thing Attributes

Things – objects and processes alike – have five basic attributes: **Perseverance**, **Essence**, **Affiliation**, **Origin**, and **Complexity**. **Perseverance** determines if the thing is an object or a process. **Essence** pertains to whether the thing is **physical** or **informatical**. **Affiliation** concerns the place where the thing belongs – the sys-

tem or the system's environment. **Origin** describes whether the thing is natural or artificial.

Perseverance is a basic thing attribute that determines whether the thing is **static** (an object) or **dynamic** (a process).

Perseverance enables distinction between objects and processes.

Essence is a basic thing attribute that determines whether the thing is **physical** or **informatical**.

The default **Essence** is **informatical**.

Affiliation is a basic thing attribute that determines whether the thing is **systemic** (part of the system) or **environmental** (external to the system).

The default **Affiliation** is **systemic**.

Origin is a basic thing attribute that determines whether the thing is **artificial** or **natural**.

To define **Complexity**, which can be simple or nonsimple, we need to first define refineables – parts, features, or specializations of a thing.

*A **refineable** of a thing T is a part of T, a feature of T, or a specialization of T.*

The thing being refined is called a refinee.

*A **refinee** is a thing that has one or more parts, features, or specializations.*

According to this definition, a refineable is a generalization of part, feature, and specialization. For example, **Wheel** is a refineable of **Car** since it is a part of **Car**, and **Car** is the refinee. Similarly, **Car** is a refineable of **Vehicle** since it is a specialization of **Vehicle**, and **Vehicle** is the refinee.

Some things have refineables while others do not. For example, an integer does not have refineables. It only has instances, such as 1, 0, 2010, etc. This distinction between things is the basis for the definition of a thing's complexity attribute.

*A thing is **simple** if it has no refineables and **nonsimple**, or compound, otherwise.*

Complexity is a basic thing attribute that determines whether the thing is **simple** or **nonsimple**.

Perseverance, **Essence**, and **Affiliation** have graphical symbols. Table 7.1 shows the entities in the OPM ontology with their graphical symbols. A thing whose perseverance is static – an object – is symbolized by a rectangle, while a thing whose perseverance is dynamic – a process – is symbolized by an ellipse. Both shapes have solid lines with no shading, indicating their default values: **informatical Essence** and **systemic Affiliation**.

A thing whose **Essence** is **physical** is symbolized by a shaded shape (rectangle or ellipse). A thing whose **Affiliation** is **environmental** is symbolized by a dashed contour. As Table 7.1 shows, any combination of a thing's **Essence** and **Affiliation** value is possible.

7.8 OPM Links

OPM links are the mortar that can connect any two OPM entities. Without links, all we could model would be a collection of isolated OPM entities with no ability to say anything about how they relate to each other structurally or procedurally. If entities are the nodes in a graph, the links are the edges. However, an OPD is more expressive than a plain mathematical graph with nodes and edges, since the entities and the links are of several types and any legal entity–link–entity combination conveys a specific defined semantics that is also translated to an English OPL sentence or part of a sentence.

Figure 7.6 extends the reflective metamodel of the OPM elements hierarchy in Fig. 7.5 with links.

At the top level, the metamodel asserts that an **OPM link** connects two **OPM Things**. At the next level, **OPM Link** specializes into **Structural Link** and **Procedural Link**.

7.8.1 Structural Links

*A **structural link** is a link that specifies the long-term, time-independent relations between two things.*

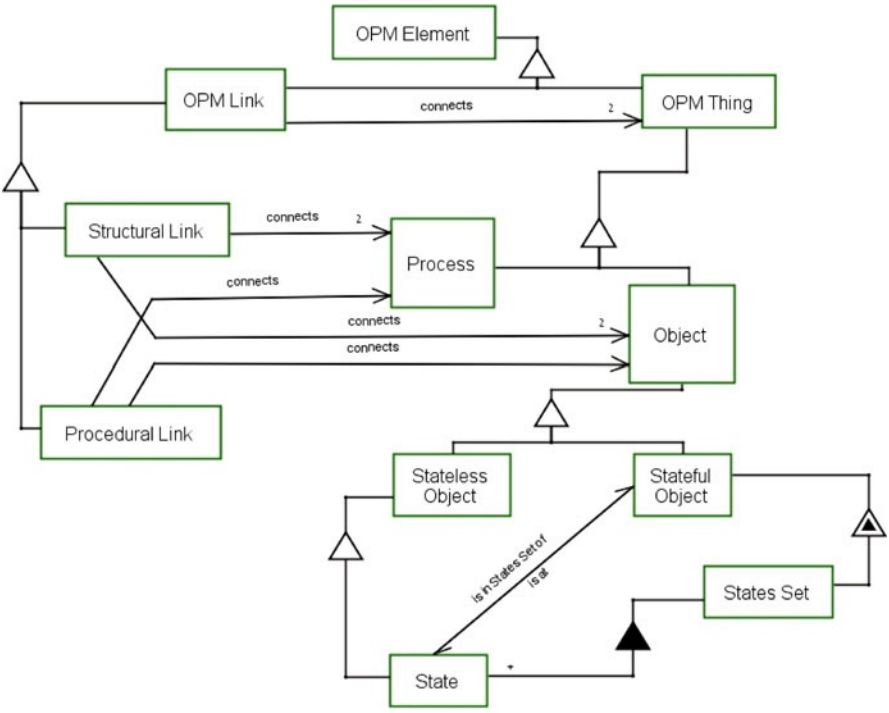


Fig. 7.6 An elaboration of the OPM reflective metamodel of the OPM elements hierarchy from Fig. 7.5 enhanced with the OPM links hierarchy

An example of a structural link in Fig. 7.6 is the generalization–specialization link from **OPM Element** to **OPM Link** and to **OPM Thing**. This generalization–specialization relation between these objects holds true and does not change over time.

As this example shows, structural links usually connect two things that have the same persistence value, namely, either two objects (things whose persistence value is static, as in this example) or two processes (things whose persistence value is dynamic). Two OPL sentences express this:

Structural Link connects 2 Objects.
Structural Link connects 2 Processes.

The only exception to this is the exhibition-characterization relation defined below.

7.8.2 Procedural Links

*A **procedural link** is a link that specifies the short-term, time-dependent relations between two things.*

Procedural links usually connect two things that have different persistence values, namely, an object and a process.

7.9 OPM Structure Modeling

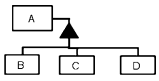
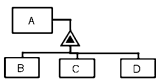
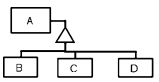
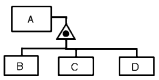
As noted, structural links graphically express static, time-independent relations between pairs of entities, most often between two objects. Structural links, shown as the middle group of six symbols in Fig. 7.4, are of two types: fundamental and tagged. Fundamental structural links are a set of four structural links that are used to denote frequently occurring relations between two entities in a system. Due to their prevalence and usefulness, and in order to prevent too much text from cluttering the diagram, these relations are designated by the four distinct triangular symbols shown in Fig. 7.4. The four fundamental structural relations are as follows:

1. **Aggregation–participation**, a solid triangle, ▲, which denotes the relation between a whole thing and its parts;
2. **Generalization–specialization**, a blank triangle, △, which denotes the relation between a general thing and its specializations, giving rise to inheritance;
3. **Exhibition–characterization**, a solid interior blank triangle, ◬, which denotes the relation between an exhibitor – a thing exhibiting one or more features (attributes or operations) – and the things that characterize the exhibitor; and
4. **Classification–instantiation**, a solid circle inside a blank triangle, ◬, which denotes the relation between a class of things and an instance of that class.

Table 7.2 lists the four fundamental structural links and their respective OPDs and OPL sentences. The name of each such relation consists of a pair of dash-separated words, e.g., *aggregation–participation*. The first word in each pair is the *forward relation* name, i.e., the name of the relation as seen from the viewpoint of the refinee – the thing up in the hierarchy that is being refined – down to its refineables. The second word is the *backward relation* name, i.e., the name of the relation as seen from the viewpoint of the refineables – the things down in the hierarchy of that relation looking up to the refinee.

Each fundamental structural link has a default, preferred direction, determined by how natural the OPL sentence that expresses its semantics sounds. In Table 7.2, the preferred shorthand name for each relation is underlined. The forward direction is the preferred one in all the links except the classification–instantiation.

Table 7.2 Fundamental structural relation names, OPD symbols, and OPL sentences

Structural Relation Name		Refinee-Refineables	OPD with 3 refineables	OPL Sentences with 1, 2, and 3 refineables
Forward	Backward			
Aggregation	Participation	Whole-Parts		**A** consists of **B.** **A** consists of **B** and **C.** **A** consists of **B, C,** and **D.**
Exhibition	Characterization	Exhibitor-Features		**A** exhibits **B.** **A** exhibits **B** and **C.** **A** exhibits **B, C,** and **D.**
Generalization	Specialization	General-Specializations		**B** is an **A.** **B** and **C** are **As.** **B, C,** and **D** are **As.**
Classification	Instantiation	Class-Instantiations		**B** is an instance of **A.** **B** and **C** are instances of **A.** **B, C,** and **D** are instances of **A.**

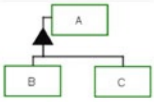
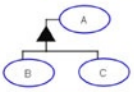
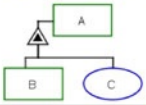
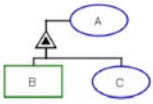
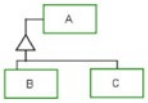
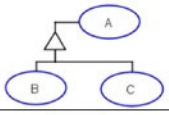
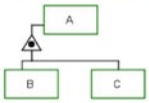
As Table 7.2 shows, each one of the four fundamental structural relations is characterized by the hierarchy it induces between the refinee – the thing attached to the tip of the triangle – and the refineables – the thing(s) attached to the base of the triangle, as follows.

1. In aggregation–participation, the tip of the solid triangle, ▲, is attached to the whole thing, while the base is attached to the parts.
2. In generalization–specialization, the tip of the blank triangle, △, is attached to the general thing, while the base is attached to the specializations.
3. In exhibition–characterization, the tip of the solid interior blank triangle, ◬, is attached to the exhibitor (the thing which exhibits the features), while the base is attached to the features (attributes and operations).
4. In classification–instantiation, the tip of the solid circle inside a blank triangle, ◬, is attached to the thing class, while the base is attached to the thing instances.

Table 7.3 presents the structural relation names, OPD symbols, OPL sentences, and semantics. It shows that there is almost symmetry between objects and processes in that for all the structural relations, whatever applies to objects also applies to processes.

Having presented the common features of the four fundamental structural relations, in the next four subsections we provide a small example of each one separately.

Table 7.3 Structural relation names, OPD symbols, OPL sentences, and semantics

STRUCTURAL LINKS				
	Name	**Symbol**	**OPL**	**Semantics**
Fundamental Structural Relations	Aggregation-Participation		**A** consists of **B** and **C**.	A is the whole, B and C are parts.
			**A** consists of **B** and **C**.	
	Exhibition-Characterization		**A** exhibits **B**, as well as **C**.	Object B is an attribute of A and process C is its operation (method). A can be an object or a process.
			**A** exhibits **B**, as well as **C**.	
	Generalization-Specialization		**B** is an **A**. **C** is an **A**.	A specializes into B and C. A, B, and C can be either all objects or all processes.
			**B** is **A**. **C** is **A**.	
	Classification-Instantiation		**B** is an instance of **A**. **C** is an instance of **A**.	Object A is the class, for which B and C are instances. Applicable to processes too.
Unidirectional & bidirectional tagged structural links			**A** relates to **B**. (for unidirectional) **A** and **C** are related. (for bidirectional)	A user-defined textual tag describes any structural relation between two objects or between two processes.

7.9.1 Aggregation–Participation

Aggregation–participation denotes the relation between a whole and its comprising parts or components. Consider, for example, the excerpt taken from Sect. 2.2 of the RDF Primer [28]:

> *. . . each statement consists of a subject, a predicate, and an object.*

This is a clear case of a whole–part, or aggregation–participation, relation. The OPM model of this statement is shown in Fig. 7.7. The OPL sentence, "**RDF Statement consists of Subject**, **Predicate**, and **Object**" was generated by OPCAT auto-

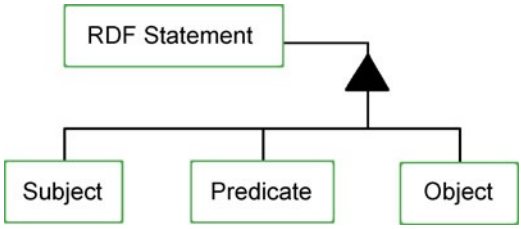


Fig. 7.7 OPD of the sentence "**RDF Statement consists of Subject**, **Predicate**, and **Object**"

matically from the graphic input and is almost identical to the one cited from the RDF Primer. The same OPD (disregarding the graphical layout) can be produced exactly by inputting the text of the OPL sentence above. This is a manifestation of the OPM graphics–text equivalence principle.

7.9.2 Generalization–Specialization

Generalization–specialization is a fundamental structural relationship between a general thing and one or more of its specializations. Continuing our example from the RDF Primer [28], consider the very first sentence from the abstract:

> *The Resource Description Framework (RDF) is a language for representing information about resources in the World Wide Web.*

Let us take the main message of this sentence, which is that *RDF is a language*. This is exactly in line with the OPL syntax, so we can input the OPL sentence "**RDF is a Language**" into OPCAT and see what we get.

The result, without any diagram editing, is shown in Fig. 7.8, along with the conversation window titled "Add new OPL sentence," in which this sentence was typed prior to the OPD creation. In fact one can justifiably argue that RDF is not a specialization of a language but rather an *instance* of a language. Indeed, instances are the leaves of the specialization hierarchy [11]. In this case we would use the classification–instantiation link, which is exemplified below.

Fig. 7.8 OPD obtained by inputting into OPCAT the OPL sentence "**RDF** is a **Language**"

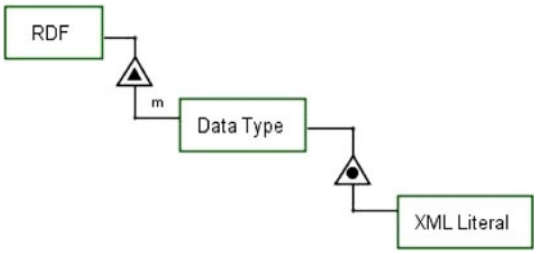


Fig. 7.9 The OPD representing the sentence "*RDF has a simple data model*"

7.9.3 Exhibition–Characterization

We continue to scan the RDF Primer [28], where in Sect. 2.2.1 we find the sentence *RDF has a simple data model*.

To model this statement, we need to rephrase this sentence into the following three sentences:

1. RDF is characterized by a data model.
2. The data model of RDF is characterized by a complexity attribute.
3. The value of this complexity attribute is "simple."

These three sentences are further rephrased to conform to the OPL syntax as follows:

1. **RDF** exhibits **Data Model**.
2. **Data Model** exhibits **Complexity**.
3. **Complexity** is **simple**.

Figure 7.9 shows the OPD representing the sentence "*RDF has a simple data model*."

7.9.4 Classification–Instantiation

Reading through the RDF Primer, we find in Sect. 3.3 on data types the following sentence:

1. *Data types are used by RDF in the representation of values, such as integers, floating point numbers, and dates.*
   ...
2. *RDF predefines just one data type, rdf:XMLLiteral, used for embedding XML in RDF.*

An OPL interpretation of the structural aspect of these two sentences, respectively, is as follows:

1. **RDF exhibits** many **Data Types**.
2. **XML Literal** is an instance of **Data Type**.

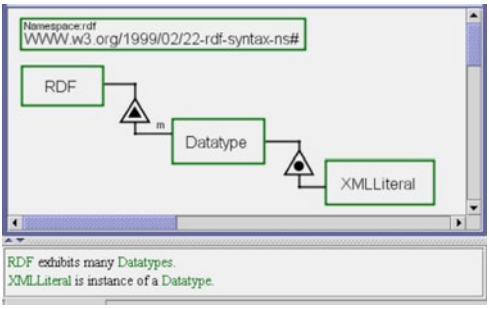


Fig. 7.10 The OPM model of **XML Literal**, an instance of a **Data Type** of **RDF**

Figure 7.10 is the OPM model of **XML Literal**, an instance of a **Data Type** of **RDF**.

7.10 OPM Behavior Modeling

Procedural links connect entities (objects, processes, and states) to express the dynamic, time-dependent behavior of the system. Behavior, the dynamic aspect of a system, can be manifested in OPM in three ways, giving rise to three types of procedural links:

1. An object can *enable* a process without being transformed by it, giving rise to *enabling links*.
2. A process can *transform* (generate, consume, or change the state of) one or more objects, giving rise to *transforming links*.
3. A thing can *control* the behavior of a system, giving rise to *control links*.

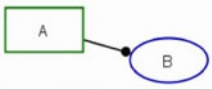
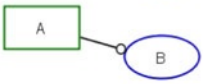
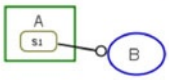
The control is done via a condition link, an event link, or an exception link. An object or a process can *trigger* an event that might, in turn, invoke a process if, as explained earlier, the process precondition is met, i.e., all the objects in the preprocess object set exist, each in its required state.

7.10.1 Enabling Links

The first group of procedural links in Table 7.4 presents the group of the three enabling links.

*An **enabler** of a process is an object that must be present, possibly at some specified state, for that process to occur.*

Table 7.4 The OPM enabling links, their OPD symbols, OPL sentences, and semantics

OPM ENABLING LINKS			
Name	Symbol	OPL	Semantics
Agent Link		A handles B.	Denotes that object A is a human operator who triggers process B.
Instrument Link		B requires A.	"Wait until" semantics: Process B cannot happen if object A does not exist.
State-Specified Instrument Link		B requires s1 A.	"Wait until" semantics: Process B cannot happen if object A is not at state s1.

*An **enabling link** is a procedural link that links an enabler to the process it enables.*

Enablers are of two types: *agents* and *instruments*.

*An **agent** is a human enabler. An **instrument** is a nonhuman enabler.*

The distinction between these two types of enablers is made primarily since humans and other objects need to be addressed differently. Humans are capable of intelligent decision making and need good interface with the system. Table 7.4 specifies the OPM enabling links, their OPD symbols, OPL sentences, and semantics. **Agent link**, the first line in Table 7.4, is a "black lollipop" – a line ending with a solid, filled-in circle.

The agent link has the semantics of a trigger – an event that attempts to launch the process to which it is attached. Indeed, humans are triggers to many high-level processes in complex systems, following which nonhuman instruments take over control of lower-level subprocesses.

An agent is just an enabler and is not supposed to be affected by the process it enables. If the human triggering a process is affected by that process, then he is no longer just an enabler but, more importantly, an affectee. For example, a person taking a medication is affected by the process of medication taking. In this case, the effect link, described below, shall be used instead of the agent link.

The agent link helps system designers to quickly locate all the places in the system where humans interact with the system, as these are places where more than just adequate human–machine interfaces will have to be designed.

Instrument link, the second line in Table 7.4, is a "white lollipop" – a line ending with a blank, white circle. An easy way to remember this difference is to think of

a human as being worth more than an instrument, so the former is filled, while the latter is empty.

The semantics of an instrument link is that of a wait-until, in the sense that when triggered, the process in question cannot start unless the instrument exists. In other words, the process is enabled if and only if the instrument exists.

The **state-specified instrument link**, shown in the bottom line of Table 7.4, is an instrument link from a specific state, s1, of object A to process B. The semantics of this link is also that of a wait-until, in the sense that when triggered, the process in question cannot start unless the instrument exists in the specified state. In other words, process B is enabled if and only if A is in state s1.

As we shall see next when discussing the transforming links, the set of two links – instrument link and its refined state-specified instrument link – is a repeating pattern of two related links: a procedural link and its state-specified refinement. At the metamodel level, we can think of the state-specified link as a specialization of the corresponding link that does not involve a state.

*A **stateless–stateful links pair** is a pair of procedural links with almost identical semantics, in which one link connects an object to a process while the other link connects an object state to a process.*

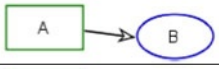
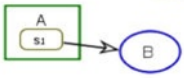
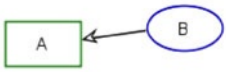
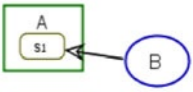
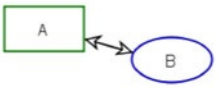
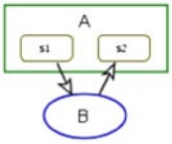
One can justifiably ask, then, what about a state-specified agent link? There is no state-specified agent link since we can always use an instrument link from a specific value or state of an attribute of the agent. For example, a voter can elect a US president if and only if she or he is a US citizen, so while the agent link will be from **Voter** to the **President Electing** process, the instrument link will be from the state **US citizen** of the attribute **Citizenship** of **Voter** to the **President Electing** process. The OPL paragraph of this system follows.

Voter exhibits Citizenship.
Citizenship can be US citizen, resident alien, or nonresident alien.
Voter handles **President Electing.**
President Electing requires **US citizen Citizenship.**
President Electing yields **President.**

7.10.2 Transforming Links

By the definition of process, a process must transform at least one object. State change is the least drastic transformation that an object can undergo. The transformation of an object can be done in one of three ways: (1) consumption of an object, (2) generation of an object, or (3) change of an object's state. Accordingly, and similar to the stateless–stateful instrument link pair discussed above, there are three pairs

Table 7.5 The OPM transforming links, their OPD symbols, OPL sentences, and semantics

OPM TRANSFORMING LINKS			
Name	Symbol	OPL	Semantics
Consumption Link		**B** consumes **A**.	Process B consumes Object A.
State-Specified Consumption Link		**B** consumes **s1 A**.	Process B consumes Object A when it is at State s1.
Result Link		**B** yields **A**.	Process B creates Object A.
State-Specified Result Link		**B** yields **s1 A**.	Process B creates Object A at State s1.
Effect Link		**B** affects **A**.	Process B changes the state of Object A; the details of the effect may be added at a lower level.
State-Specified Effect Link (*Input-Output Links Pair*)		**B** changes **A** from **s1** to **s2**.	Process B changes the state of Object A from State s1 to State s2.

of stateless–stateful transforming links, giving rise to the six transforming links in Table 7.5.

1. The pair consisting of a consumption link and its refined state-specified consumption link has the semantics of consumption. Referring to the first two lines in Table 7.5, the consumption link denotes that occurrence of process B eliminates object A. The state-specified consumption link denotes that the occurrence of process B eliminates object A, provided that object A is at state s1.
2. The pair consisting of a result link and its refined state-specified result link has the semantics of generation or creation. Referring to the two middle lines in Table 7.5, the result link denotes that the occurrence of process B yields object A. The state-specified result link denotes that the occurrence of process B yields object A at state s1.
3. The pair consisting of an effect link and its refined state-specified effect link has the semantics of state change. Referring to the two bottom lines in Table 7.5, the effect link denotes that the occurrence of process B changes the state of object A. The state-specified effect link, also called an *input–output links pair*, is a pair of two links in opposite directions, denoting that the occurrence of process B changes the state of object A from its input state s1 to its output state s2.

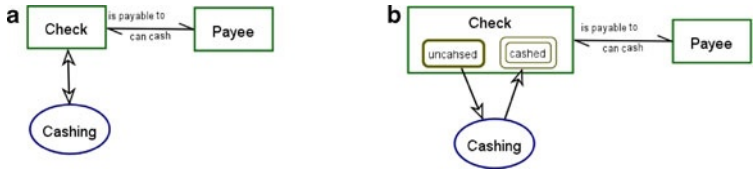


Fig. 7.11 Effect link (**a**) and state-specified effect link (**b**). The effect link denotes that **Cashing** changes the state of **Check**. The state-specified effect link denotes that **Cashing** changes **Check** from **uncashed** to **cashed**

Figure 7.11 exemplifies the effect link in Fig. 7.11a and the state-specified effect link in Figure 7.11b. We can think of the effect link as an abstraction of its refined version. Sometimes we may not be interested in specifying the states of an object but still show that a process does affect an object by changing its state from some unspecified input state to another unspecified output state. To express this, we suppress (hide) the input and output states of the object, so the edges of the input and output links "migrate" to the contour of the object and coincide, yielding the bidirectional effect link as a superposition of two opposite-pointing arrows.

An effect is object transformation in which a process changes the state of an object from some input state to another output state. When these two states are expressed (i.e., shown explicitly), as in the OPD in Fig. 7.11b, then we can use the pair of input and output links to specify the source and destination states of the transformation. When the states are suppressed (hidden), we express the state change by the effect link, a more general and less informative transformation link.

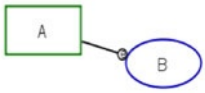
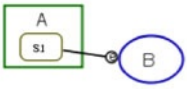
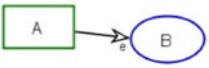
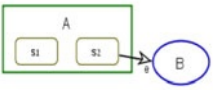
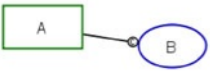
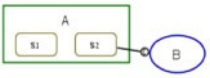
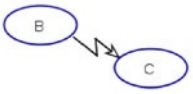
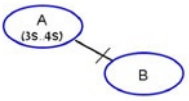
The two more extreme transformations are generation and consumption, denoted respectively by the result and consumption links. Generation is a transformation that causes an object that had not existed prior to the process execution to become existent. In contrast to generation, consumption is a transformation that causes an object that had existed prior to the process execution to become nonexistent.

7.10.3 Control Links

The OPM control links (Table 7.6) control the dynamics of the system, expressed as the conditions and order of flow of processes and the associated object transformations that take place as the system becomes operational. Most OPM control links are combinations of a procedural link with an event or a condition.

The first stateless–stateful links pair is the instrument event link and its refined *state-specified instrument event link*. The **instrument event link** combines the semantics of the instrument link with the semantics of event. Referring to the first line in Table 7.6, the creation of object A is an event that triggers process B. B will start executing if its precondition (i.e., all the objects in the preprocess object set exist, and are in their required states, if so specified) is met. Since A is an instrument, it will not be affected by B.

Table 7.6 OPM control links, their OPD symbols, OPL sentences, and semantics

OPM CONTROL LINKS			
Name	**Symbol**	**OPL**	**Semantics**
Instrument Event Link		**A** triggers **B**. **B** triggers **A**.	Generation of object A is an event that triggers process B. B will start executing if its precondition is met. Since A is instrument it will not be affected by B.
State-Specified Instrument Event Link		A triggers **B**. when it enters **s1**. **B** requires **s1** **A**.	Entering state s1 of object A is an event that triggers process B. B will start executing if its precondition is met. Since A is instrument it will not be affected by B.
Consumption Event Link		**A** triggers **B**. **B** consumes **A**.	Generation of object A is an event that triggers process B. B will start executing if its precondition is met, and if so it will consume A.
State-Specified Consumption Event Link		**A** triggers **B** when it enters **s2**. **B** consumes **s2** **A**.	Entering state s2 of A is an event that triggers process B. If B is triggered, it will consume A. B will start executing if its precondition is met, and if so it will consume A.
Condition Link		**B** occurs if **A** exists.	Existence of object A is a condition for the execution of B. If A does not exist, then B is skipped and regular system flow continues.
State-Specified Condition Link		**B** occurs if **A** is **s1**.	Existence of object A at state s2 is a condition for the execution of B. If A is not in s2, then B is skipped and regular system flow continues.
Invocation Link		**B** invokes **C**.	Execution termination of process B is an event that triggers process C. B yields a temporary object that is immediately consumed by C and therefore not shown explicitly in the model.
Exception Link		**A** triggers **B** when it lasts more than 4 seconds.	Process A has to be assigned with maximal acceptable time duration, which, if exceeded, triggers process B.

The **state-specified instrument event link** combines the semantics of the state-specified instrument link with the semantics of event. Referring to the second line in Table 7.6, entering state s1 of object A is an event that triggers process B. B will start executing if its precondition is met, again without affecting B since A is an instrument.

The second stateless–stateful links pair is the *consumption event link* and its refined *state-specified consumption event link*.

The **consumption event link** combines the semantics of the consumption link with the semantics of event. Referring to the third line in Table 7.6, the creation of object A is an event that triggers process B. B will start executing if its precondition is met, in which case it will consume B.

The **state-specified consumption event link** combines the semantics of the state-specified consumption link with the semantics of event. Referring to the fourth line in Table 7.6, entering state s2 of object A is an event that triggers process B. B will start executing if its precondition is met, in which case it will consume B.

The third stateless–stateful links pair is the *condition link* and its refined *state-specified condition link*. Referring to the fifth line in Table 7.6, the semantics of the **condition link** is that the existence of object A is a condition for the execution of B. If A does not exist, then B is skipped and regular system flow continues.

Referring to the sixth line in Table 7.6, the semantics of the **state-specified condition link** is that the existence of object A in state s2 is a condition for the execution of B. If A does not exist in s2, then B is skipped and regular system flow continues.

A fourth stateless–stateful links pair is the *consumption condition link* and its refined *state-specified consumption condition link*. This pair, which does not appear in Table 7.6, is graphically similar to the consumption event link and its refined *state-specified consumption event link*, except that the letter c appears instead of the letter e. The semantics of this pair is similar to the condition link and its refined state-specified condition link, except that *if B occurs, it consumes A*.

The last two control links, shown in the two bottom lines in Table 7.6, are the invocation link and the exception link. They are exceptional among the procedural links, as unlike the other procedural links, which connect an object with a process, they connect two processes. The **invocation link** semantics is that execution termination of process B is an event that triggers process C. B yields a temporary object that is immediately consumed by C and therefore not shown explicitly in the model. The **exception link** semantics requires that process A be assigned with maximal acceptable time duration, which, if exceeded, triggers process B. This is a way to avoid indefinite waiting for a process that, for some reason, got stuck and delays further execution of the system. C is an exception handling process, which has to be designed to take care of the failure of B to terminate within the time allotted for its execution by taking a remedial action or notifying a human.

7.11 Complexity Management

Complexity is inherent in real-life systems. An integral part of a system development methodology must therefore include control and management of this complexity. Like most classical engineering problems, complexity management entails a trade-off that must be balanced between two conflicting requirements: completeness and clarity. On one hand, *completeness* requires that the system details be stipulated in the model to the fullest extent possible. On the other hand, the need for *clarity* implies that no diagram of the model be too cluttered or overloaded. This can be

achieved by imposing an upper limit on the level of complexity of each individual diagram. OO development methods, notably the UML standard [30] and its SysML derivative, address the problem of managing systems complexity by dividing the system model into different views for the various aspects of the system – structure, dynamics, event sequence, physical architecture, state transitions, etc.

The approach OPM takes is orthogonal, advocating the integration of the various system aspects into a single model. Rather than applying a separate model for each system aspect, OPM handles the inherent system complexity by introducing a number of abstraction–refinement mechanisms. These enable presenting and viewing the things that comprise the system at various detail levels. The entire system is completely specified through its OPD set – a hierarchical set of interconnected OPDs that together provide a full picture of the system being investigated or developed. Along with the OPD set goes the automatically generated OPL system specification. This section elaborates on these complexity management issues and specifies the various abstracting–refining mechanisms.

Complexity is managed in OPM via three refinement–abstraction mechanisms: in-zooming and out-zooming, unfolding and folding, and state expression and suppression. These mechanisms allow for looking at any complex system at any desired level of granularity without losing the context and the "big picture." We elaborate on complexity management in this section.

7.11.1 The Need for Complexity Management

The very need for systems analysis and design strategies stems from complexity. If systems or problems were simple enough for humans to grasp by merely glancing at them, no methodology would be required. Due to the need for tackling sizeable, complex problems, a system development methodology must be equipped with a comprehensive approach, backed by a set of reliable and useful tools, for controlling and managing this complexity. This challenge entails balancing two forces that pull in opposite directions and need to be traded off: completeness and clarity. *Completeness* means that the system must be specified to the last relevant detail. *Clarity* means that to communicate the analysis and design outcomes, the documentation, be it textual or diagrammatic, must be legible and comprehensible. To tackle complex systems, a methodology must be equipped with adequate tools for complexity management that address and solve this problem of completeness–clarity tradeoff by striking the right balance between these two contradicting demands.

OPM achieves clarity through abstracting and completeness through refining. Abstracting, the inverse of refining, saves space and reduces complexity, but it comes at the price of completeness. Conversely, refining, which contributes to completeness, comes at the price of loss of clarity. There are "no free meals"; as is typically the case with engineering problems, there is a clear tradeoff between completeness of details and clarity of their presentation. The solution OPM proposes is to keep each OPD simple enough and to distribute the system specification over

a set of consistently interrelated OPDs that contain things at various detail levels. Abstracting and refining are the analytical tools that provide for striking the right balance between clarity and completeness.

Analysis and design are the first steps in the lifecycle of a new system, product, or project. Creating (sometimes unconscious) resistance on the side of the prospective audience to accept the analysis and design results, because they look too complex and intimidating, may have an adverse effect of jeopardizing the success of subsequent phases of the product development. The severity and frequency of this detail explosion problem calls for an adequate solution to meet the needs of the systems analysis community. A major test of any analysis methodology is therefore the quality of tools for managing the ever-growing complexity of analysis outcomes in a coherent, clear, and useful manner. Such complexity management tools are extremely important for organizing the knowledge the architect accumulates and generates during the system architecting process. Equally important is the role of complexity management mechanisms in facilitating the communication of the analysis and design results to stakeholder, including customers, peers, superiors, and system developers down the development cycle road – implementers, testers, and users.

7.11.2 Middle-Out as the De Facto Architecting Practice

Analyzing is the process of gradually increasing the human analyzer's knowledge about a system's structure and behavior. Designing is the process of gradually increasing the amount of details about the system's architecture, i.e., the structure and behavior combination that enables the system to attain its function. For both analysis and design, managing the system's complexity therefore entails being able to present and view the system at various levels of detail that are consistent with each other. Ideally, analysis and design start at the top and make their way gradually to the bottom – from the general to the detailed. In real life, however, analysis typically starts at some arbitrary detail level and is rarely linear. The design is not linear either. Almost invariably, these are iterative processes, during which knowledge, followed by understanding, is gradually accumulated and refined in the conceptual model.

The system architect often cannot know in advance the precise structure and behavior of the very top of the system – this requires analysis and becomes apparent at some point along the analysis process. Step by step, the analyst builds the system specification by accumulating and recording facts and observations about things in the system and relations among them. Using OPM, the accumulated knowledge is represented through a set of OPDs and their corresponding OPL paragraphs. The sheer amount of details contained in any real-world system of reasonable size overwhelms the system architect soon enough during the architecting process. Trying to incorporate the details into one diagram, the amount of drawn symbols gets very large, and their interconnections quickly become an entangled web. For all but the simplest systems, this information overload happens even if the method advocates using multiple diagram types for the various system aspects. Because the diagram

has become so cluttered, it is increasingly difficult to comprehend it. System architects experience this detail-explosion phenomenon on a daily basis, and anyone who has tried to analyze a system will endorse this description. The problem calls for effective and efficient tools to manage this inherent complexity.

Due to the nonlinear nature of these processes, unidirectional "bottom-up" or "top-down" approaches are rarely applicable to real-world systems. Rather, it is frequently the case that the system under construction or investigation is so complex and unexplored that neither its top nor its bottom is known with certainty from the outset. More commonly, analysis and design of real-life systems start in an unknown place along the system's detail-level hierarchy. The analysis proceeds "*middle-out*" by combining top-down and bottom-up techniques to obtain a complete understanding and specification of the system at all the detail levels. It turns out that even though architects usually strive to work in an orderly top-down fashion, more often than not, the de facto practice is the middle-out mode of analysis and design. Rather than trying to fight it, we should build software tools that provide facilities to handle this middle-out architecting mode. Such facilities cater also to both top-down and bottom up approaches.

During the middle-out analysis and design process, facts and ideas about objects in the system and its environment, and processes that transform them, are being gathered and recorded. As the analysis and design proceed, the system architect tries to *concurrently* specify both the structure and the behavior of the system in order to enable it to fulfill its function. For an investigated (as opposed to an architected) system, the researcher tries to make sense of the long list of gathered observations and to understand their cause and effect relation. In both cases, the system's structure and behavior go hand in hand, and it is very difficult to understand one without the other. Almost as soon as a new object is introduced into the system, the process that transforms it or is enabled by it begs to be modeled as well. By supplying the single object–process model, OPM caters to this structure–behavior concurrency requirement. It enables modeling these two major system aspects at the same time within the same model without the need to constantly switch between different diagram types.

If the OPD that is being augmented becomes too crowded, busy, or unintelligible, a new OPD is created. This descendant OPD repeats one or more of the things in its ancestor OPD. These repeated things establish the link between the ancestor and descendant OPDs. The descendant OPD does not usually replicate all the details of its ancestor, as some of them are abstracted, while others are simply not included. This new OPD is therefore amenable to refinement of new things to be laid out in the space that was saved by not including things from the ancestor OPD. In other words, there is room in it to insert a certain amount of additional details before it gets too cluttered again. When this happens, a new cycle of refinement takes place, and this goes on until the entire system has been completely specified.

7.11.3 The Completeness-Comprehension Dilemma

We face a dilemma here, called the *completeness-comprehension dilemma*, which is typical in conceptual modeling. The dilemma is that on one hand we wish to continue adding facts to the system under study or design, but on the other hand we do not want to compromise the clarity of the graphics by overcluttering it. This dilemma pops up regardless of the conceptual modeling language used – it is simply an unavoidable outcome of details starting to pile up that require and compete for diagram "real estate."

A balance needs to be struck and a tradeoff has to be found between completeness of details on the one hand and keeping the model legible on the other. The solution OPM offers to solve this problem is dividing the model into a set of separate yet logically integrated and hierarchically organized OPDs via a couple of refinement mechanisms. The mechanism we have used is *in-zooming*. In-zooming creates a new, descendant OPD and provides for refining the blown-up process by modeling its subprocesses and their interactions with lower-level objects. Another OPM refinement mechanism is unfolding, in which refineables of a thing are linked using one of the fundamental structural links in a new OPD or in an existing one.

7.12 Applications and Standardization of OPM

OPM has been applied in a variety of domains, including real-time systems [32], Web-based systems [35, 43], Enterprise Resource Planning [15, 39], systems architecture [38], web service composition [51], Product Lifecycle Engineering [14], molecular biology [13], data warehouse construction [12], privacy management in medical records [4], software reuse and design patterns [36, 37], domain analysis [41], exceptions modeling [33], intelligent house [52], and multiagent systems [42].

The domain-independent nature of OPM makes it suitable as a general, comprehensive, and multidisciplinary framework for knowledge representation and reasoning that emerge from conceptual modeling, analysis, design, implementation, and lifecycle management. The ability of OPM to provide comprehensive lifecycle support for systems of all kinds and complexity levels is due to its foundational ontology that builds on a most minimal set of stateful objects and processes that transform them. Another significant trait of OPM is its unification of system knowledge from both the structural and behavioral aspects in a single model expressed diagrammatically via the OPD set and textually via the set of corresponding OPL paragraphs. OPM features dual knowledge representation in graphics and text, so users have the capability to automatically switch between these two modalities. It is hard to think of a significant domain of discourse and a system within it where structure and behavior are not interdependent and intertwined. Due to its single model, expressed in both graphics and text, OPM lends itself naturally to representing and managing knowledge, as it is uniquely positioned to cater to the tight interconnections between structure and behavior that are so hard to separate, making it a most

suitable language for structure–behavior codesign. The ability to model physical and informatical things also makes OPM ideal for hardware–software codesign.

OPM is not just a language but also a system development and lifecycle-support methodology, for which a comprehensive reflective metamodel (which uses OPM) has been developed [34]. Indeed, in a 2008 survey by INCOSE – the International Council on Systems Engineering [18] – OPM was recognized as one of the six leading model-based systems engineering (MBSE) methodologies, which include also IBM Telelogic Harmony-SE, INCOSE Object-Oriented Systems Engineering Method, IBM Rational Unified Process for Systems Engineering for Model-Driven Systems Development, Vitech Model-Based System Engineering Methodology, and JPL State Analysis.

Several activities are being undertaken to leverage the potential benefits of OPM in international standardization bodies. Work is under way by ISO TC184/SC5 OPM Working Group to develop a Draft International Standard (DIS) for OPM that will be the basis for authoring model-based enterprise standards and other technical documents [20, 24, 25]. This DIS is expected to be presented in the ISO TC184/WG5 annual meeting in the USA in 2011.

References

1. American Heritage Dictionary of the English Language (1996) Houghton Mifflin, Boston, 4th edn
2. Baddeley A (1992) Working memory. Science 255:556–559
3. Beckhoff B, Kanngießer B, Langhoff N, Wedell R, Wolff H (2006) Handbook of practical x-ray fluorescence analysis. Springer, Berlin Heidelberg New York
4. Beimel D, Peleg M, Dori D, Denekamp Y (2008) Situation-based access control: privacy management via patient data disclosure modeling. J Biomed Inform 41(6):1028–1040
5. Bunge MA (1977) Treatise on basic philosophy, vol 3: Ontology I: the furniture of the world. Reidel, Boston
6. Bunge MA (1979) Treatise on basic philosophy, vol 4. Ontology II: a world of systems, Reidel, Boston
7. Chandler P, Sweller J (1991) Cognitive load theory and the format of instruction. Cogn Instr 8:293–332
8. Clark JM, Paivio A (1991) Dual coding theory and education. Educ Psychol Rev 3:149–210
9. Dodaf (2007) http://cio-nii.defense.gov/docs/DoDAF_Volume_I.pdf
10. Dori D (1995) Object-process analysis: maintaining the balance between system structure and behavior. J Log Comput 5(2):227–249
11. Dori D (2002) Object-Process Methodology: a holistic systems paradigm. Springer, Berlin Heidelberg New York
12. Dori D (2008) Words from pictures for dual channel processing. Commun ACM 51(5):47–52
13. Dori D, Choder M (2007) Conceptual modeling in systems biology fosters empirical findings: the mRNA lifecycle. In: Proceedings of the library of science ONE (PLoS ONE), September 2007.
    http://www.plosone.org/article/info
14. Dori D, Shpitalni M (2005) Mapping knowledge about product lifecycle engineering for ontology construction via object-process methodology. CIRP Ann Manuf Technol 54(1):117–122
15. Dori D, Golany B, Soffer P (2005) Aligning an ERP system with enterprise requirements: an object-process based approach. Comput Ind 56(6):639–662

16. Dori D, Feldman R, Sturm A (2008) From conceptual models to schemata: an object-process based data warehouse construction method. Inf Syst 33:567–593
17. Dori D, Reinhartz-Berger I, Sturm A (2003) Developing complex systems with object-process methodology using OPCAT. In: Proceedings of ER 2003. Lecture notes in computer science, vol 2813. Springer, Berlin, pp 570–572
18. Estefan J (2008) Survey of model-based systems engineering (MBSE) methodologies. http://www.incose.org/productspubs/pdf/techdata/MTTC/\discretionary-MBSE_\discretionary-Methodology_Survey_2008-0610_RevB-JAE2.pdf
19. Glenberg AM, Langston WE (1992) Comprehension of illustrated text: pictures help to build mental models. J Mem Lang 31:129–151
20. Howes DB, Blekhman A, Dori D (2010) Model based verification and validation of a manufacturing and control standard. In: Proceedings of Manufacturing and Service Operations Management Society (MSOM) annual conference, Maalot, Israel, June 2010
21. Hegarty M (1992) Mental animation: inferring motion from static displays of mechanical systems. J Exp Psychol Learn Mem Cogn 18:1084–1102
22. Hayes P, Menzel C (2001) A semantics for the knowledge interchange format. In: IJCAI 2001 workshop on the IEEE Standard Upper Ontology
23. IDEF (2001) A structured approach to enterprise modeling and analysis. www.idef.com. IDEF Family of Methods
24. ISO N1049 (2009) OPM study group, terms of reference. Plenary Meeting Resolutions 2009-04-23/24
25. ISO-N1078 ISO/TC 184/SC 5 (2010) Plenary Meeting Resolutions 2010-03-25/26
26. Mayer RE (2003) The promise of multimedia learning: Using the same instructional design methods across different media. Learn Instr 13:125–139
27. Mayer RE, Moreno R (2003) Nine ways to reduce cognitive load in multimedia learning. Educ Psychol 38(1):43–52
28. Manola F, Miller E (2004) RDF primer. http://www.w3.org/TR/rdf-primer
29. Miller GA (1956) The magical number seven, plus or minus two: Some limits on our capacity for processing information. Psychol Rev 63:81–97
30. OMG: Object Management Group (2010) http://www.omg.org
31. Ontology Markup Language Version 0.3. http://www.ontologos.org/OML/OML
32. Peleg M, Dori D (2000) The model multiplicity problem: Experimenting with real-time specification methods. IEEE Trans Softw Eng 26(8):742–759
33. Peleg M, Somekh J, Dori D (2009) A methodology for eliciting and modeling exceptions. J Biomed Inform 42(4):736–747
34. Reinhartz-Berger I, Dori D (2005) A reflective metamodel of object-process methodology: the system modeling building blocks. Idea Group, Hershey, PA, pp 130–173
35. Reinhartz-Berger I, Dori D, Katz S (2002) OPM/Web – object-process methodology for developing web applications. Ann Softw Eng 13:141–161
36. Reinhartz-Berger I, Dori D, Katz S (2009) Reusing semi-specified behavior models in systems analysis and design. J Softw Syst Model 8:221–234
37. Shlezinger G, Reinhartz-Berger I, Dori D (2010) Modeling design patterns for semi-automatic reuse in system design. J Database Manag 21(1):29–57
38. Soderborg N, Crawley E, Dori D (2003) OPM-based system function and architecture: definitions and operational templates. Commun ACM 46(10):67–72
39. Soffer P, Golany B, Dori D (2003) ERP modeling: a comprehensive approach. Inf Syst 28(6):673–690
40. Sowa JF (2000) Knowledge representation: logical, philosophical, and computational foundations. Brooks Cole, Pacific Grove, CA
41. Sturm A, Dori D, Shehory O (2009) Application-based domain analysis approach and its object-process methodology implementation. Int J Softw Eng Knowl Eng 19(1)
42. Sturm A, Dori D, Shehory O (2010) An object-process-based modeling language for multi-agent systems. IEEE Trans Syst Man Cybern C Appl Rev 40(2):227–241
43. Toch E, Gal A, Reinhartz-Berger I, Dori D (2007) A semantic approach to approximate service retrieval. ACM Trans Internet Technol 8(1):2:1–2:30

44. University of Arizona (2001) NATS-online. http://www.ic.arizona.edu/~nats101/n1.html
45. Von Glaserfeld E (1987) The construction of knowledge: contributions to conceptual semantics. Intersystems, Seaside, CA
46. Wand Y, Weber R (1989) An ontological evaluation of systems analysis and design methods. Elsevier, North Holland, pp 145–172
47. Wand Y, Weber R (1993) On the ontological expressiveness of information systems analysis and design grammars. J Inf Syst 3:217–237
48. Wand Y, Storey VC, Weber R (1999) An ontological analysis of the relationship construct in conceptual modeling. ACM Trans Database Syst 24(4):494–528
49. Webster's Encyclopedic Unabridged Dictionary of the English Language (1984) Portland House, New York
50. Webster's New Dictionary (1997) Promotional Sales Books
51. Yin L, Wenyin L, Changjun J (2004) Object-process diagrams as explicit graphic tool for web service composition. J Integr Des Process Sci Trans SDPS 8(1):113–127
52. Zoref L, Bregman D, Dori D (2009) Networking mobile devices and computers in an intelligent home. Int J Smart Home 3(4):15–22